Equilibrium-projected graph neural network surrogates for linear frame analysis: uncertainty-aware FE referral and residual-guided adaptive learning

Karthik Research[a,*]

[a] *Affiliation to be completed, Country*

Abstract

Data-driven surrogates of finite element (FE) models can accelerate repeated structural assessment, but their member forces usually violate equilibrium and they give no indication of when they should not be trusted. This paper develops equilibrium-projected graph neural network surrogates for the first-order linear analysis of planar moment frames, with every analysis run in the open-source PyNite library and verified against an independent stiffness solver, OpenSeesPy and closed-form solutions. A graph network ensemble predicts all nodal displacements and member forces in a dimensionless, load-case-separated form, so that superposition, homogeneity in the loads, the $1/E$ scaling of displacements and the boundary conditions hold exactly. A projection in a variance-weighted metric maps the predicted forces to the nearest field in static equilibrium. Two indicators are used for two different decisions: at deployment, a domain check on geometry and load-pattern descriptors followed by a split-conformal bound on the ensemble spread decides which frames are referred back to the FE solver; during training, the equilibrium residual of the raw prediction decides which frames receive new FE analyses. On 9,500 PyNite analyses and five independent training runs, the median end-moment error was $2.37 \pm 0.03\%$ on in-distribution frames and $1.99 \pm 0.08\%$ on design-informed frames, four to nine times lower than five tabular baselines given the same features; a StructGNN-style network reached similar accuracy, and displacement-only networks with force recov-

*Corresponding author. Author names and affiliations are placeholders to be completed by the authors before submission.

Email address: karthikresearchworks@gmail.com (Karthik Research)

ery were much less accurate. On unseen geometries and load patterns the median moment errors rose to 7.5% and 8.7%, with 95th percentiles above 20%. The projection reduced nodal out-of-balance forces from 4–16% to below 10^{-12} and the axial-force errors by factors of 2–5, but changed the moment errors only modestly. With its domain threshold and conformal quantile calibrated on two separate sets of new frames, the referral rule reached the guaranteed marginal coverage of the end-moment error on in-distribution test frames (observed 90.7% for a nominal 90%); it accepted 99% of them, with an observed, not guaranteed, 1.4% of the accepted frames above a 10% end-moment error, and referred 71–73% of unseen frames. The rule is calibrated for the end-moment field only, and drift and displacement errors of accepted frames are not covered. The predicted forces are in equilibrium but not compatible with the predicted displacements, so the referral framework supports member-force screening, the kinematic outputs serve only as supplementary screening estimates, and the surrogate is not a complete structural analysis. Residual-guided acquisition reached the accuracy of 1,200 random analyses on unseen geometries with 600 analyses, while variance-guided acquisition was worse than random sampling.

Keywords: Graph neural network, Surrogate model, Static equilibrium, Deep ensemble, Conformal prediction, Out-of-distribution detection, Active learning, Frame structures

1. Introduction

Many tasks in structural engineering require the same structural system to be analysed many times: preliminary sizing and design-space exploration, code-based checks over large numbers of load combinations and variants, portfolio-scale assessment of existing buildings, optimisation and reliability analysis. Each analysis is cheap in isolation, but the number of repetitions makes finite element (FE) analysis the dominant cost of these loops. Surrogate models, which replace the FE solver by a fast approximation learned from a limited number of FE runs, have therefore a long history in engineering design [1] and reliability analysis [2, 3], and machine learning (ML) has substantially widened the range of quantities that can be emulated [4].

For structural assessment, however, a surrogate is only useful if its output can be trusted in the way an FE result is trusted. Three obstacles stand in the way. First, purely data-driven models carry no guarantee that their

outputs are mechanically admissible: predicted member forces need not be in equilibrium with the applied loads, the predicted response need not scale linearly with the loads in a linear analysis, and boundary conditions are only approximately respected. An engineer who checks the reactions against the loads will immediately notice such defects. Second, accuracy reported on test data drawn from the training distribution says little about frames whose geometry or loading differ from those seen in training, which is the normal situation in practice. Third, the cost of generating training data is often treated as fixed, although the FE budget is usually the scarcest resource, and the surrogate itself does not indicate when its prediction should not be used.

Physics-informed machine learning [5] addresses the first obstacle by embedding physical knowledge in the learning model. Physics-informed neural networks penalise the residual of the governing equations in the loss [6] and have been applied to solid mechanics [7]; in structural dynamics, physics-guided convolutional and recurrent networks have been used to model the seismic response of buildings [8, 9]. A penalty term, however, only encourages the constraint and leaves a residual violation whose size depends on the training. Constraints can instead be imposed exactly, either through the network output layer [10] or through hard-constraint formulations [11]. Graph neural networks (GNNs) [12, 13] are a natural representation for structures whose topology varies: they have been used to simulate particle and mesh-based physics [14, 15], to predict stress and deformation fields [16] and to build reusable surrogates for truss structures [17]. On the side of uncertainty, deep ensembles [18] provide a simple estimate of epistemic uncertainty, although its quality degrades under distribution shift [19]; and adaptive sampling driven by surrogate uncertainty is well established for Kriging models in structural reliability [2, 3] and more broadly as active learning [20].

1.1. Recent developments in mechanics-informed graph learning for structural analysis

Graph representations have become the default choice for learning the static response of skeletal structures, because joints and members map directly onto nodes and edges and the same network can be applied to structures of different size. Table 1 compares the studies closest to the present one; entries marked "n.r." were not reported in the text we could access, and the comparison is limited to what each paper states.

Whalen and Mueller [17] trained a graph surrogate on linear FE results of parametric trusses to predict nodal displacements from geometry, supports and loads. Their model is purely data-driven (mean-squared-error loss), and they reported that it did not generalise well to topologies excluded from training, which motivated transfer learning. Maurizi et al. [16] predicted stress, strain and deformation fields of materials and structures with a message-passing network, again without mechanics constraints. Parisi et al. [21] proposed a mechanics-informed surrogate model (MISM) in which the structural topology enters through the graph and the section properties through edge features of an edge-conditioned convolution. It predicts the nodal displacements and rotations of two- and three-dimensional frame and truss structures, but uses a plain mean-absolute-error loss with no equilibrium or compatibility term, and was tested on a random split of the same generator. Chou et al. [22] introduced StructGNN for the static analysis of multi-storey buildings. Its graph adds one pseudo node per storey, connected to all joints of that floor as a proxy for a rigid diaphragm, and uses as many message-passing layers as storeys plus one so that information can travel the full building height. Trained on about 2,000 buildings of two to six storeys, it predicted displacements and member forces with high accuracy and kept useful accuracy on taller seven- to nine-storey buildings. It does not enforce equilibrium and gives no uncertainty estimate. Beyond static frame analysis, graph surrogates are being developed for other structural response problems. Liang et al. [23] proposed PeTIGN, a physics-encoded graph network for computing the dynamic response of large-scale structures, and Patrignani and Pinho [24] combined local neighbourhood attention with periodic global attention in a graph transformer to predict strain fields of composite plates and woven composites, addressing the long-range interactions that local message passing captures poorly. Both target different response quantities and structural domains (dynamic response of large structures; continuum strain fields) and are therefore not benchmarked here, but they show the same trend towards physical priors and long-range information exchange that motivates the global-pooling and StructGNN-style comparisons of this study. Kuo et al. [25] used message-passing networks to return displacements, shear forces and bending moments of multi-storey frames under wind for interactive design and reported promising generalisation to taller configurations. Most recently, Leng et al. [26] proposed a mechanically constrained GNN for planar frames with member loads. Their network predicts only the nodal displacements, adds the free-DOF stiffness residual and global equilibrium residuals

to the loss as soft penalties, and recovers member forces from the displacements through the element stiffness relations with a fixed-end correction for member loads. Displacement-based recovery makes the forces compatible with the predicted displacements, but equilibrium then holds only to the extent that the penalty is minimised.

Three gaps remain. First, no study in Table 1 enforces nodal equilibrium exactly; where it appears at all, it is a soft penalty. Second, none quantifies predictive uncertainty or uses it to decide when to fall back on the FE solver or where to spend new FE analyses. Third, out-of-distribution testing, where present, has concerned taller buildings only; loading patterns outside the training distribution have not been examined. These remain open in the broader graph-surrogate literature cited above as well, whose physical priors are built into the network rather than imposed on its output. The present study addresses the gaps for linear static planar frames through an exact equilibrium projection of the predicted forces, a referral rule whose domain and conformal thresholds are calibrated on separate data, and residual-based adaptive acquisition of FE analyses, and it compares directly against re-implementations of the StructGNN-style and Leng-style designs (Section 5).

1.2. Objectives and contributions

This paper develops and evaluates, for planar moment-resisting frames under first-order linear static analysis, a workflow in which (a) the predicted member forces satisfy nodal static equilibrium exactly rather than on average, (b) the known linear structure of the problem (superposition, homogeneity in the loads, scaling with Young's modulus, boundary conditions) is imposed rather than learned, (c) predictive uncertainty is used at deployment to refer doubtful predictions back to the FE solver and an equilibrium residual is used during training to choose new FE analyses, and (d) the result is tested against conventional regression, machine-learning and graph-learning baselines on geometries and load patterns outside the training distribution. The open-source PyNite FE library [27] generates all data; an independent stiffness solver and OpenSeesPy [28] verify it. The research question is whether a carefully validated surrogate can predict displacements and member forces accurately enough to reduce repeated FE analysis in screening tasks, with FE analysis retained for final verification. The surrogate is called equilibrium-projected because exact static equilibrium of the member forces (together with exact linear scaling) is the only mechanical condition it enforces; compatibility between the predicted displacements and forces is measured but not

Table 1: Mechanics-informed graph learning for the static analysis of skeletal structures. Eq.: equilibrium enforcement; Compat.: compatibility enforcement; UQ: uncertainty quantification; AS: adaptive sampling; OOD: out-of-distribution test. n.r.: not reported in the accessible text.

Study	Structural system	Architecture	Predicted quantities	Eq.	Compat.	UQ	AS	OOD testing
Whalen and Mueller 2022 [17]	Trusses	Graph surrogate, transfer learning	Nodal displacements	None	None	No	No	Unseen topologies, loads (poor without transfer)
Maurizi et al. 2022 [16]	Continua, lattices	Message passing	Stress, strain, deformation fields	None	None	No	No	n.r.
Parisi et al. 2024 [21]	2D/3D frames, trusses	Edge-conditioned convolution (MISM)	Nodal displacements, rotations	None (MAE loss)	None	No	No	No (random split)
Chou et al. 2024 [22]	Multi-storey buildings	StructGNN: storey pseudo nodes, depth $n_s + 1$	Displacements, member forces	None	None	No	No	Taller buildings (7–9 storeys)
Kuo et al. 2025 [25]	Multi-storey frames, wind	Message passing	Displacements, shears, moments	n.r.	n.r.	n.r.	n.r.	Taller frames
Leng et al. 2026 [26]	Planar frames, member loads	Displacement GNN + force recovery	Displacements; forces recovered	Soft (residual loss)	By construction (recovery)	No	No	n.r.
This study	Planar moment frames	Edge/node GNN ensemble + projection	Displacements, basic forces, reactions	Exact (projection)	Measured, not enforced	Ensemble + conformal	Yes	Taller/wider frames; new load patterns; design-informed set

enforced (Section 3.6), so the surrogate is not a complete structural analysis.

The contributions are:

1. A graph-network surrogate in which load-case superposition, homogeneity in the load intensities, the $1/E$ scaling of displacements and the boundary conditions hold exactly by construction, with exact statics quantities and classical approximations supplied as inputs.
2. An equilibrium projection that maps any predicted member-force field to the nearest statically admissible field in a variance-weighted metric, with an error-reduction guarantee in that metric (Proposition 1) and an analysis of its effect on displacement–force compatibility.
3. A two-stage deployment rule, a domain check on the frame's geometry and load-pattern descriptors followed by a split-conformal threshold on the ensemble spread, both calibrated on data separate from the test sets, and a residual-guided adaptive FE sampling scheme evaluated over five repetitions.
4. A benchmark of 9,500 PyNite analyses, verified against two independent solvers and closed-form solutions, with in-distribution,

unseen-geometry, unseen-load-pattern and design-informed test sets; comparisons with five tabular baselines and five graph-learning baselines, including StructGNN-style and Leng-style re-implementations; and statistics over five independent training runs. The code, data generator, frame parameters and trained weights have been prepared as a reproducibility package (see Data and code availability); its archived public release is pending.

The remainder of the paper is organised as follows. Section 2 describes the frame family, the PyNite data generation and its independent verification, Section 3 the surrogate and the constraints, Section 4 the uncertainty-guided workflow and Section 5 the baselines and metrics. Results are presented in Section 6 and discussed, with the limitations of the study, in Section 7. Section 8 concludes.

2. Finite element data generation with PyNite

2.1. Parametric frame family

The study considers planar, multi-storey, multi-bay moment-resisting frames with rigid beam–column joints, a family that is routinely re-analysed during preliminary design and assessment. A frame is defined by the number of storeys n_s and bays n_b, the bay widths L_b, the storey heights h_s, Young's modulus E, the second moments of area of the columns ($I_{c,s}$) and beams ($I_{b,s}$) in each storey, the corresponding cross-sectional areas $A = I/r^2$ obtained from a sampled radius of gyration r, the base condition (fixed or pinned), the uniformly distributed gravity load $w_{s,b}$ on each beam and a lateral force F_s at each floor. All columns of a storey share one section and column stiffness may decrease with height through a taper factor. Units are kN and m throughout. The sampling ranges are listed in Table 2; E spans concrete-like to steel-like values so that the surrogate cannot rely on a fixed material.

Three scenarios are generated (Fig. 3):

- *Training design space (ID).* $1 \leq n_s \leq 5$, $1 \leq n_b \leq 4$; the gravity load is uniform along each floor but varies between floors; the lateral profile is either uniform or inverted-triangular, in either direction.

- *Unseen geometries (OOD-G).* Frames with 6–8 storeys and/or 5–6 bays, and in half of the cases a tall first storey (4.5–6.0 m), all outside the training ranges.

- *Unseen loading (OOD-L).* Training geometries subjected to pattern gravity loading, in which the live part of the load is randomly removed from individual bays, and to lateral profiles that were never seen in training (parabolic, or a single force at the roof).

- *Design-informed synthetic frames (DES).* 300 steel moment frames whose parameters are correlated through simplified sizing rules instead of being sampled independently: $E = 200\,\mathrm{GPa}$; bays of 6.0, 7.5 or 9.0 m; a taller first storey (4.0–4.5 m) over typical storeys (3.2–3.8 m); gravity loads from $1.2D + 1.0L$ area loads on a 4–6 m tributary width; wind forces from a uniform facade pressure of 0.6–1.2 kPa. Beams are sized for an $L/360$ live-load deflection and a simple-span moment demand, columns for an $h/400$ storey drift under wind (shear-building estimate with a beam-flexibility factor of 0.5) with one section per two storeys, and 80% of the bases are fixed. Section properties are clipped to the training ranges (28% of the frames have at least one clipped column section), so only the parameter correlations and load proportions are new; line loads reach 52 kN/m, above the training range, which the load scaling of Section 3.2 makes irrelevant. The set shares the topology family of the training data, the sizing rules are not code-compliant designs, and 29% of the frames exceed the $h/400$ target in the FE analysis (Section 6); it is therefore a design-informed synthetic benchmark, not a set of real or code-checked designs. The complete generation procedure, including the sizing equations and clipping, is given in Appendix C.

The OOD-G set tests extrapolation in topology and size, the OOD-L set extrapolation in the load pattern, and the DES set a design-informed synthetic benchmark with practice-like parameter correlations; none is used for training, model selection or early stopping. The sampling procedure is listed step by step in Appendix B.

2.2. PyNite analyses and statics verification

Each frame is built as an `FEModel3D` in PyNite [27] (version 3.2.0 of the fork cited) with one Euler–Bernoulli member per column and beam segment. Out-of-plane translations and rotations are restrained at every node so that the 3D solver reproduces planar behaviour. The lateral and gravity loads are defined as two separate load cases, W and G, and solved by a single

Table 2: Sampling ranges of the frame family. $\mathcal{U}$: uniform, $\log\mathcal{U}$: log-uniform. Units: kN, m.

Parameter	Training design space (ID)	Unseen scenarios
Storeys n_s	$\{1,\dots,5\}$	OOD-G: $\{6,7,8\}$ or $n_b \in \{5,6\}$
Bays n_b	$\{1,\dots,4\}$	OOD-G: up to 6
Bay width L_b	$\mathcal{U}(3,9)$	same
Storey height h_s	$\mathcal{U}(2.8,4.5)$	OOD-G: first storey $\mathcal{U}(4.5,6.0)$ in 50%
Young's modulus E (kN/m^2)	$\log\mathcal{U}(2.5\times10^7, 2.1\times10^8)$	same
Column I_c at base (m^4)	$\log\mathcal{U}(5\times10^{-5}, 2\times10^{-3})$, taper $\mathcal{U}(0.5,1)$	same
Beam $I_{b,s}$ (m^4)	$\log\mathcal{U}(5\times10^{-5}, 2\times10^{-3})$ per storey	same
Radius of gyration r	$\mathcal{U}(0.08,0.30)$ (columns, beams)	same
Base condition	fixed (70%) / pinned (30%)	same
Gravity load w (kN/m)	$\mathcal{U}(5,40)$ per floor, all bays	OOD-L: 30–60% of w in random bays
Lateral profile	uniform or inverted triangle	OOD-L: parabolic or roof-only
Roof lateral force (kN)	$\mathcal{U}(5,80)$, random sign	same

first-order linear analysis (`analyze_linear`). For each case the nodal displacements (u_x, u_y, θ_z) and the global member end forces are extracted; the latter are converted to the basic forces of Section 3.1. The wall-clock time of the complete PyNite call chain (model construction, analysis and result extraction) is recorded for every frame and later used for the cost comparison.

As a guard against convention errors, every FE result is checked against an independent implementation of the frame statics (Section 3.1): the out-of-balance nodal force computed from the PyNite member forces must vanish. All 9,500 analyses pass with residuals at round-off level (below 10^{-6} of the largest applied load; typically 10^{-11} kN). The data comprise a candidate pool of 5,000 ID frames, 400 ID validation frames, three test sets of 600 frames each (ID, OOD-G, OOD-L), the 300 DES frames and, for the referral rule only, two calibration sets of 1,000 ID frames each (Section 3.8), all generated with disjoint random seeds.

2.3. Independent verification of the reference solutions

Because every error reported below is measured against PyNite, the reference data were verified independently in two ways. First, all 2,100 test frames were re-analysed with two solvers that share no code with PyNite: a direct-stiffness solver for planar Euler–Bernoulli frames written for this study

Table 3: Independent verification of the PyNite reference solutions. Top: largest relative L_2 difference per frame between each solver and PyNite over every frame of each test set. Bottom: closed-form benchmarks analysed with PyNite.

Test set	Solver	u_x	u_y	θ_z	Reactions	Column M	Beam M	Shear V	Axial N
ID	Own DSM	5.8e-12	4.4e-12	5.5e-12	4.5e-12	5.8e-12	6.1e-12	6.1e-12	4.4e-12
	OpenSeesPy	4.8e-12	3.8e-12	4.5e-12	3.9e-12	4.8e-12	5.2e-12	5.2e-12	3.8e-12
OOD-G	Own DSM	1.2e-11	5.3e-12	1.1e-11	4.3e-12	1.3e-11	1.0e-11	1.3e-11	5.2e-12
	OpenSeesPy	8.2e-12	3.0e-12	7.4e-12	2.7e-12	9.0e-12	7.3e-12	9.1e-12	3.0e-12
OOD-L	Own DSM	3.9e-12	1.4e-12	4.5e-12	2.1e-12	4.1e-12	4.1e-12	4.0e-12	1.6e-12
	OpenSeesPy	3.3e-12	1.4e-12	3.0e-12	1.3e-12	3.2e-12	3.2e-12	3.2e-12	1.4e-12
DES	Own DSM	3.7e-13	1.2e-14	3.0e-13	1.9e-14	1.7e-13	7.6e-14	1.5e-13	1.5e-14
	OpenSeesPy	2.7e-13	2.4e-14	2.2e-13	3.0e-14	1.2e-13	8.8e-14	1.2e-13	2.7e-14

Closed-form benchmarks (PyNite / exact)
Portal, rigid beam, fixed bases: sway 4.4662 / 4.4661 mm; base moment 43.750 / 43.750 kN m
Portal, rigid beam, pinned bases: sway 17.8647 / 17.8646 mm; top moment 87.500 / 87.500 kN m
Beam with fixed ends, uniform load: end moment 60.0000 / 60.0000 kN m

(vectorised element matrices, consistent fixed-end forces for the member loads, sparse direct solution with SciPy), and OpenSeesPy [28] (version 3.8.0; `elasticBeamColumn` elements with a linear transformation, `beamUniform` element loads, UMFPACK solver). Table 3 lists the largest relative L_2 difference per frame between each solver and PyNite for horizontal and vertical displacements, rotations, support reactions, column and beam end moments, end shears and axial forces. All differences are below 1.3×10^{-11}, i.e. at round-off level. Second, PyNite was run on three classical cases with closed-form solutions: a portal frame with a rigid beam under a lateral load P with fixed bases (sway $Ph^3/(24EI_c)$, column end moments $Ph/4$) and with pinned bases (sway $Ph^3/(6EI_c)$, top moments $Ph/2$), and a beam with fixed ends under a uniform load (end moments $wL^2/12$). The PyNite results differ from the closed-form values by at most 9×10^{-6} in relative terms, which is explained by the finite stiffness of the very stiff members used to represent the rigid idealisations.

3. Equilibrium-projected graph neural network surrogate

The surrogate maps the frame description to the full response field: all nodal displacements and the basic forces of all members. Three kinds of mechanics are built into it, in decreasing order of strictness: (i) exact constraints that hold for any network output (boundary conditions, homogeneity in the load intensities, nodal equilibrium after projection); (ii) mechanics-derived inputs that are exact statical quantities or classical approximations; and

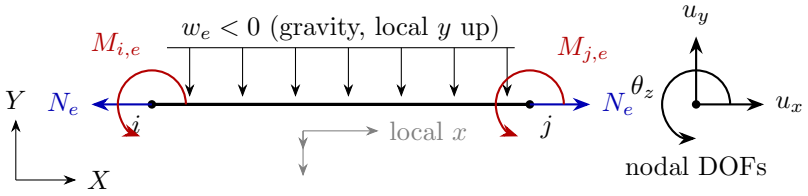


Figure 1: Sign conventions. Basic forces of member e ($i \to j$): axial force N_e, tension positive; end moments $M_{i,e}$, $M_{j,e}$ acting on the member, counter-clockwise positive; transverse load w_e along the local y axis. Nodal displacements u_x, u_y along the global axes and rotation θ_z counter-clockwise. Columns are oriented bottom to top, beams left to right.

(iii) an inductive bias in the architecture (message passing on the structural graph).

3.1. Statics of the frame and the equilibrium constraint

For member e between nodes i and j with length L_e and direction cosines (c_e, s_e), the basic forces are $\mathbf{q}_e = [N_e, M_{i,e}, M_{j,e}]^\mathsf{T}$: the axial force (tension positive) and the two end moments acting on the member, counter-clockwise positive. Columns run from bottom to top and beams from left to right; nodal displacements (u_x, u_y, θ_z) are positive along global X, Y (upwards) and counter-clockwise (Fig. 1). For a member carrying a transverse distributed load w_e, the six local end forces are

$$\mathbf{f}_e = \mathbf{b}_e \mathbf{q}_e + \mathbf{f}_e^0, \qquad \mathbf{b}_e = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1/L_e & 1/L_e \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & -1/L_e & -1/L_e \\ 0 & 0 & 1 \end{bmatrix}, \tag{1}$$

where w_e is the local transverse load intensity (positive along the local y axis, so gravity on a beam is negative) and $\mathbf{f}_e^0 = -\frac{1}{2} w_e L_e \, [0, 1, 0, 0, 1, 0]^\mathsf{T}$ is the simply supported particular solution.

This parametrisation satisfies member equilibrium identically [29]. Rotating to global axes and assembling at the nodes gives the nodal equilibrium equations at the n_f free degrees of freedom (DOFs),

$$\mathbf{C}\,\mathbf{q} = \mathbf{d}, \qquad \mathbf{C} \in \mathbb{R}^{n_f \times 3m}, \quad \mathbf{d} = \mathbf{P}_f - \mathbf{p}_f^0, \tag{2}$$

where $\mathbf{q}$ stacks the basic forces of all m members, $\mathbf{P}_f$ are the applied nodal loads and $\mathbf{p}_f^0$ the assembled particular solutions. $\mathbf{C}$ is the equilibrium (statics) matrix. For a kinematically stable frame it has full row rank and $3m-n_f$ equals the degree of static indeterminacy, so Eq. (2) defines an affine set of dimension $3m - n_f > 0$ that contains the exact FE solution but also every other statically admissible force field. Equilibrium alone therefore cannot determine the forces, which is precisely why a learned prior is needed; conversely, any force prediction that violates Eq. (2) is mechanically inadmissible. Support reactions follow from $\mathbf{q}$ at the restrained DOFs, so a force field satisfying Eq. (2) also satisfies global equilibrium. Assembling $\mathbf{C}$ costs $\mathcal{O}(m)$ and requires no stiffness matrix.

3.2. Dimensional analysis and exact load-case superposition

For a linear elastic frame the response is linear in the loads. If E is the same in every member, as here, the displacements and rotations are inversely proportional to E, whereas the member forces of this statically indeterminate structure depend only on the relative stiffnesses and are therefore independent of a uniform E. The surrogate imposes these properties exactly through dimensional analysis [30]. Let $H = \frac{1}{n_s}\sum_s h_s$ be the mean storey height and $I_{\mathrm{ref}} = \frac{1}{n_s}\sum_s I_{c,s}$ the mean column second moment of area, so that EI_{ref} is a reference flexural rigidity. Each load case $c \in \{\mathrm{W}, \mathrm{G}\}$ is scaled by its own intensity: the signed base shear $\lambda_{\mathrm{W}} = \sum_s F_s$ and the total gravity load $\lambda_{\mathrm{G}} = \sum_e |w_e| L_e$. The response is written as

$$
\begin{aligned}
(u_x, u_y) &= \sum_c \frac{\lambda_c H^3}{EI_{\mathrm{ref}}} \, (\hat{u}_{x,c}, \hat{u}_{y,c})(\boldsymbol{\pi}_c), \quad & \theta_z &= \sum_c \frac{\lambda_c H^2}{EI_{\mathrm{ref}}} \, \hat{\theta}_c(\boldsymbol{\pi}_c), \\
N &= \sum_c \lambda_c \, \hat{N}_c(\boldsymbol{\pi}_c), & M &= \sum_c \lambda_c H \, \hat{M}_c(\boldsymbol{\pi}_c),
\end{aligned}
\tag{3}
$$

where hats denote the network outputs and $\boldsymbol{\pi}_c$ is a set of dimensionless input groups (Table 4): lengths divided by H, second moments of area divided by I_{ref}, areas as AH^2/I_{ref}, lateral loads divided by λ_{W} and gravity loads by λ_{G}. Because $\boldsymbol{\pi}_c$ is invariant to the load magnitudes and to E, Eq. (3) is exactly homogeneous of degree one in each load-case intensity, the displacements scale exactly with $1/E$, the forces are exactly independent of a uniform E, and reversing the lateral load reverses the lateral response. E does not appear in the force expressions at all, and λ_c cancels in every input group.

Table 4: Dimensional consistency of the scaled formulation (units kN, m). Every input and output group of the network is dimensionless and independent of λ_c and E.

Quantity	Units	Scale	Dimensionless group
Lengths x, L, h	m	H	x/H, L/H, h/H
Second moment of area I	m^4	I_{ref}	I/I_{ref}
Cross-sectional area A	m^2	I_{ref}/H^2	AH^2/I_{ref}
Floor force F_s, storey shear V_s	kN	λ_{W}	F_s/λ_{W}, V_s/λ_{W}
Overturning moment	kN m	$\lambda_{\mathrm{W}}H$	$\mathrm{OTM}/(\lambda_{\mathrm{W}}H)$
Line load w (resultant wL)	kN/m (kN)	λ_{G}	wL/λ_{G}
Drift estimate δ_s	m	$\lambda_{\mathrm{W}}H^3/EI_{\mathrm{ref}}$	$\delta_s EI_{\mathrm{ref}}/(\lambda_{\mathrm{W}}H^3)$
Displacements u_x, u_y	m	$\lambda_c H^3/EI_{\mathrm{ref}}$	$\hat{u}_{x,c}$, $\hat{u}_{y,c}$
Rotation θ_z	rad	$\lambda_c H^2/EI_{\mathrm{ref}}$	$\hat{\theta}_c$
Axial force N	kN	λ_c	$\hat{N}_c$
End moment M	kN m	$\lambda_c H$	$\hat{M}_c$
Equilibrium rows (force / moment)	kN / kN m	λ_c / $\lambda_c H$	rows of $\mathbf{C}$, $\mathbf{d}_c$

In the generator, $|\lambda_{\mathrm{W}}| \geq 5\,\mathrm{kN}$ (all floor forces of a frame have the same sign) and $\lambda_{\mathrm{G}} > 0$, so neither scale can vanish. In deployment a load case with $\lambda_c = 0$ is simply omitted from the sum; a lateral pattern whose floor forces cancel ($\sum_s F_s \approx 0$ but $\sum_s |F_s| > 0$) would make λ_{W} ill-conditioned and should be scaled by $\sum_s |F_s|$ instead, a case that does not occur in this study and that the domain check of Section 3.8 would flag as unseen. Predicting the two cases separately also avoids a scaling problem: in the combined case the lateral sway is often orders of magnitude smaller than the gravity-induced quantities, so a single scale under-weights it. The ablation in Section 6.4 quantifies this choice. Restrained DOFs are multiplied by zero, so the boundary conditions also hold exactly.

3.3. Mechanics-informed features

Each node carries 25 dimensionless features and each member 10. Beyond position, topology and section ratios, they include two quantities from elementary statics that are exact and cost $\mathcal{O}(n)$, the storey shear $V_s = \sum_{k \geq s} F_k$ and the overturning moment of the loads above the storey, and three classical approximations. The first is the gravity load carried by each column line computed from tributary areas, i.e. as if every beam were simply supported; in a rigid frame continuity and sway redistribute these loads, so the feature is an approximation, not an exact statical quantity. The other two are the beam-to-column stiffness ratio of each storey, $\rho_s = (n_b I_{b,s}/\bar{L}_b)/((n_b+1)I_{c,s}/h_s)$, and

the shear-building drift estimate $\delta_s = V_s/k_s$ with $k_s = (n_b+1)\,12EI_{c,s}/h_s^3$ (or $3EI/h^3$ for a pinned first storey), together with its height-wise cumulative sum. The network learns the correction from these simple estimates to the FE response, which is a smoother target than the response itself. The full feature dictionary with formulas is given in Appendix A. The same features are given to all tabular baselines.

3.4. Graph network

Frames of different size and topology are represented as graphs whose nodes are joints and whose edges are members, and processed with an encode–process–decode message passing network [12, 13, 14, 15]. Node and edge features are encoded by two-layer perceptrons (width 96, SiLU, layer normalisation). Each of the eight processing steps updates every edge from its own state and the states of its end nodes, $\mathbf{z}_e \leftarrow \mathbf{z}_e + \phi_e([\mathbf{z}_e, \mathbf{h}_i, \mathbf{h}_j])$, and every node from the sums of the messages of the members that start and end at it, kept separate so that the network can distinguish the two ends of a member, $\mathbf{h}_i \leftarrow \mathbf{h}_i + \phi_n([\mathbf{h}_i, \sum_{e:i=\text{start}} \mathbf{z}_e, \sum_{e:i=\text{end}} \mathbf{z}_e])$. Eight steps nearly cover the graph diameter of the training frames (at most nine members), and frame-level information is supplied in addition through the statics features; the effect of the depth (4, 8, 12 and 16 steps) is examined in Section 6.4. Decoders output six dimensionless displacements per node and six dimensionless basic forces per member (three per load case). The loss is the mean squared error of the outputs standardised per component, over free DOFs only. Training uses AdamW [31] with a one-cycle learning-rate schedule (peak 2×10^{-3}), batches of 32 frames, gradient clipping at 1.0 and 150 epochs, implemented in PyTorch [32]. A variant with a soft equilibrium penalty $\|\mathbf{C}\hat{\mathbf{q}}_c - \mathbf{d}_c\|^2$ added to the loss is examined in the ablation.

3.5. Equilibrium projection

A data-driven prediction $\hat{\mathbf{q}}$ will in general violate Eq. (2). The proposed constraint layer replaces it with the closest statically admissible force field in a metric $\boldsymbol{\Sigma}^{-1}$:

$$\mathbf{q}^\star = \arg\min_{\mathbf{C}\mathbf{q}=\mathbf{d}} (\mathbf{q}-\hat{\mathbf{q}})^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\mathbf{q}-\hat{\mathbf{q}}) = \hat{\mathbf{q}} + \boldsymbol{\Sigma}\mathbf{C}^\mathsf{T}\left(\mathbf{C}\boldsymbol{\Sigma}\mathbf{C}^\mathsf{T}\right)^{-1}(\mathbf{d}-\mathbf{C}\hat{\mathbf{q}}), \quad (4)$$

which follows from the Karush–Kuhn–Tucker conditions of the equality-constrained quadratic programme [33]. It is applied separately to each load case in dimensionless form. Two metrics are compared: the Euclidean metric

on the dimensionless forces ($\boldsymbol{\Sigma} = \mathbf{I}$), and the *variance-weighted* metric built from the ensemble variance σ_k^2 of each predicted dimensionless force component k (Section 3.7), so that the components the model is least sure of absorb most of the correction. Because an ensemble variance can be arbitrarily close to zero, the metric is regularised as

$$\boldsymbol{\Sigma}_{\mathrm{reg}} = \mathrm{diag}\big(\sigma_k^2 + \varepsilon\,\bar{\sigma}^2\big)\,, \qquad \bar{\sigma}^2 = \frac{1}{3m}\sum_k \sigma_k^2, \quad \varepsilon = 10^{-6}, \tag{5}$$

per load case and frame; the floor makes $\boldsymbol{\Sigma}_{\mathrm{reg}}$ positive definite and scale-free. The system $(\mathbf{C}\boldsymbol{\Sigma}\mathbf{C}^{\mathsf{T}})\boldsymbol{\mu} = \mathbf{d} - \mathbf{C}\hat{\mathbf{q}}$ is solved by a dense LU factorisation with partial pivoting (LAPACK `gesv` through NumPy), so there is no iterative tolerance. Because this solve, like the assembly of the features and of $\mathbf{C}$, is carried out in NumPy outside the automatic-differentiation graph of the network, the implemented pipeline is not differentiable end to end with respect to the frame parameters (Section 6.10); the resulting equilibrium residuals are at round-off level (Section 6). The sensitivity of the results to ε, to the alternative floor $\mathrm{diag}(\max(\sigma_k^2, \varepsilon\bar{\sigma}^2))$ and the condition numbers of $\mathbf{C}\boldsymbol{\Sigma}\mathbf{C}^{\mathsf{T}}$ are reported in Section 6.7. If the prediction error were Gaussian, unbiased and independent between components with covariance $\boldsymbol{\Sigma}$, Eq. (4) would be the posterior mean after conditioning on the equilibrium equations. The ensemble variance is at best a crude diagonal approximation of that covariance (the errors of neighbouring members are correlated and the ensemble underrepresents the error under distribution shift), so this interpretation motivates the choice of metric but does not make $\mathbf{q}^\star$ a calibrated posterior.

Proposition 1. *Let $\mathbf{q}_{\mathrm{FE}}$ be the exact solution, so that $\mathbf{C}\mathbf{q}_{\mathrm{FE}} = \mathbf{d}$. For any prediction $\hat{\mathbf{q}}$ and any symmetric positive definite $\boldsymbol{\Sigma}$, the projected forces satisfy $\|\mathbf{q}^\star - \mathbf{q}_{\mathrm{FE}}\|^2_{\boldsymbol{\Sigma}^{-1}} = \|\hat{\mathbf{q}} - \mathbf{q}_{\mathrm{FE}}\|^2_{\boldsymbol{\Sigma}^{-1}} - \|\hat{\mathbf{q}} - \mathbf{q}^\star\|^2_{\boldsymbol{\Sigma}^{-1}} \le \|\hat{\mathbf{q}} - \mathbf{q}_{\mathrm{FE}}\|^2_{\boldsymbol{\Sigma}^{-1}}$.*

Proof. $\mathbf{q}^\star$ is the $\boldsymbol{\Sigma}^{-1}$-orthogonal projection of $\hat{\mathbf{q}}$ onto the affine set $\mathcal{A} = \{\mathbf{q} : \mathbf{C}\mathbf{q} = \mathbf{d}\}$. Since $\mathbf{q}^\star - \mathbf{q}_{\mathrm{FE}}$ lies in $\ker\mathbf{C}$ and $\hat{\mathbf{q}} - \mathbf{q}^\star = \boldsymbol{\Sigma}\mathbf{C}^{\mathsf{T}}\boldsymbol{\mu}$ for some $\boldsymbol{\mu}$, their $\boldsymbol{\Sigma}^{-1}$-inner product is $\boldsymbol{\mu}^{\mathsf{T}}\mathbf{C}(\mathbf{q}^\star - \mathbf{q}_{\mathrm{FE}}) = 0$, and the identity follows from Pythagoras' theorem. □

The projection can therefore never increase the error measured in its own metric, and it removes exactly the part of the error that is visible to statics. It does not guarantee a reduction of every individual component (for example the end moments alone), which is why the choice of metric matters

and is examined empirically. The projection needs one dense solve of size n_f per load case, which for the frames considered here takes a fraction of a millisecond, and it is model-agnostic: it is applied below to the baselines as well.

3.6. Displacement–force compatibility

The network predicts displacements and forces with separate decoders, and the projection changes only the forces. The two fields are therefore not linked by the element stiffness relations. To measure this, the basic forces implied by the predicted displacements are computed member by member as $\mathbf{q}_{\text{const}}(\hat{\mathbf{u}})$, i.e. the local end forces $\mathbf{k}_e\mathbf{T}_e\hat{\mathbf{u}}_e + \mathbf{f}_e^{\text{fix}}$ (element stiffness $\mathbf{k}_e$, rotation $\mathbf{T}_e$ and fixed-end forces $\mathbf{f}_e^{\text{fix}}$ of the member load), and the compatibility residual is defined as

$$r_{\text{compat}} = \frac{\|\mathbf{S}(\mathbf{q} - \mathbf{q}_{\text{const}}(\hat{\mathbf{u}}))\|_2}{\|\mathbf{S}\,\mathbf{q}_{\text{const}}(\hat{\mathbf{u}})\|_2},$$
$$r_{\text{compat},M} = \frac{\|\mathbf{M} - \mathbf{M}_{\text{const}}(\hat{\mathbf{u}})\|_2}{\|\mathbf{M}_{\text{const}}(\hat{\mathbf{u}})\|_2}, \qquad r_{\text{compat},N} = \frac{\|\mathbf{N} - \mathbf{N}_{\text{const}}(\hat{\mathbf{u}})\|_2}{\|\mathbf{N}_{\text{const}}(\hat{\mathbf{u}})\|_2}, \tag{6}$$

where $\mathbf{q}$ is the force field being assessed (raw, projected or recovered) and $\mathbf{S}$ divides the moment entries by H so that forces and moments are commensurate. $r_{\text{compat},M}$ and $r_{\text{compat},N}$ use the end moments and the axial forces only; they are reported separately because the axial forces implied by displacements, EA/L times the member elongation, amplify small displacement errors by the large axial stiffness. A zero residual means that the forces are those the predicted displacements would produce; the exact FE solution has $r_{\text{compat}} = 0$ and zero equilibrium residual simultaneously. Recovering the forces from the predicted displacements, as in displacement-based surrogates [26], gives $r_{\text{compat}} = 0$ by construction but sacrifices equilibrium; projecting the directly predicted forces gives exact equilibrium but leaves compatibility unenforced. Both options, and the effect of the projection on r_{compat}, are quantified in Section 6.6. A field that satisfies both conditions exactly is the FE solution itself; the predicted displacements can then serve as the initial iterate of an iterative solver, which is also examined.

3.7. Predictive uncertainty

Epistemic uncertainty [34] is estimated with a deep ensemble [18] of $M = 5$ networks that differ in initialisation and batch order and share the

training data and normalisation. The ensemble mean is the prediction and the ensemble variance σ^2 of each output feeds the projection metric. For decisions at the level of a whole frame, the relative spread of the end moments,

$$s = \frac{\|\boldsymbol{\sigma}_M\|_2}{\|\hat{\mathbf{M}}\|_2}, \tag{7}$$

is used, where $\boldsymbol{\sigma}_M$ collects the physical standard deviations of all end moments. A second, label-free indicator is also examined: the equilibrium residual $\|\mathbf{C}\hat{\mathbf{q}} - \mathbf{d}\|/\|\mathbf{d}\|$ of the unprojected ensemble mean, which measures how far the raw prediction is from being statically admissible. The two indicators have different roles in the workflow. The ensemble spread s is used at deployment, to decide whether an individual prediction is accepted or referred to the FE solver; the equilibrium residual is used during training, to decide which candidate frames are worth an FE analysis (Section 4).

3.8. Domain check and calibrated referral threshold

An ensemble can be confidently wrong on inputs unlike its training data [19], and Section 6.8 shows that this happens for the unseen load patterns. The deployment rule therefore has two stages. To keep the statistical argument clean, each quantity of the rule is fitted or calibrated on its own data, and none of these data is used for testing or for training the networks. Besides the training frames, two additional sets of in-distribution frames are generated for this purpose with their own seeds and are used for nothing else: a *domain-calibration set* of 1,000 frames and a *conformal-calibration set* of 1,000 frames (the 400 validation frames, which were used for model selection, are not used by the rule). The procedure has four steps.

Step A: domain model. Each frame is summarised by ten geometry descriptors (numbers of storeys and bays, largest storey height, first-to-mean storey height ratio, mean bay width, ratio of the largest to the smallest bay, height-to-width aspect ratio, $\log_{10} E$, $\log_{10}$ of the beam-to-column second-moment ratio, base fixity) and five load-pattern descriptors (normalised height of the centroid of the lateral forces, roof share of the lateral load, mean ratio of each beam load to the largest load on its floor, mean within-floor coefficient of variation of the beam loads, and $\log_{10}$ of the gravity-to-lateral load ratio). All are available before any analysis. The descriptors are standardised with the training-set statistics, and the Mahalanobis distance [35] $D(\mathbf{x})$ to the training distribution is computed with a Ledoit–Wolf shrinkage covariance [36] estimated on the 3,000 training frames.

Step B: domain threshold. The threshold D^* is the $\lceil (n_\mathrm{B}+1)(1-\alpha_\mathrm{D}) \rceil$-th smallest distance of the $n_\mathrm{B} = 1{,}000$ domain-calibration frames, with $\alpha_\mathrm{D} = 0.01$. A frame with $D > D^*$ is outside the validated domain and is referred to FE. Because the domain model is fixed before these frames are seen, a new in-distribution frame satisfies $D \leq D^*$ with probability at least $1 - \alpha_\mathrm{D}$ (split-conformal argument for the score D). Distances based on the geometry and the load descriptors alone are also reported, to show which group detects which shift.

Step C: conformal error bound. With D^* fixed, the in-domain frames of the conformal-calibration set are selected ($D \leq D^*$), and for them the non-conformity scores $a_i = e_{M,i}/s_i$ (true end-moment error over ensemble spread, Eq. (7)) are computed. $\hat{a}$ is their $\lceil (n_\mathrm{C}+1)(1-\alpha) \rceil$-th smallest value, with $\alpha = 0.1$ and n_C the number of selected frames. The bound for a new frame is $\hat{a}\, s$.

Step D: decision. A frame is accepted if it is inside the domain and $\hat{a}\, s \leq \tau$ for a tolerance τ on the relative end-moment error (5% and 10% are reported), and referred otherwise. Both thresholds are fixed before the test sets are used, and the rule is evaluated only on the untouched test sets. Steps C and D are repeated for each of the five independently trained ensembles.

What is guaranteed. Because the domain rule is fixed independently of the conformal-calibration frames, selecting frames with it preserves exchangeability: for a new in-distribution frame that passes the domain check, $P(e_M \leq \hat{a}\, s) \geq 1 - \alpha$, marginally over the calibration and test frames [37]. Since a frame is accepted only if $\hat{a}\, s \leq \tau$, it follows that $P(\text{accepted and } e_M > \tau) \leq \alpha$ for such frames. The guarantee concerns the joint probability; the fraction of *accepted* frames with $e_M > \tau$ (the conditional violation rate) is not bounded by the theory and is reported as an empirical quantity. For the shifted test sets (OOD-G, OOD-L) and for the DES set the exchangeability assumption does not hold, and all coverage and violation rates reported for them are empirical observations, not guarantees. Finally, the bound concerns only the relative L_2 error of the end-moment field, e_M. It gives no guarantee for other quantities, such as the largest end moment, axial forces, drifts or displacements; for these, Section 6.8 reports how often the accepted frames exceed the same tolerance, as separately validated estimates.

4. Uncertainty-guided workflow

The surrogate is embedded in a workflow with two uses of the uncertainty estimate (Fig. 2).

Adaptive FE data generation. Training starts from N_0 randomly chosen frames of the candidate pool, analysed with PyNite. In each round an ensemble is trained, every remaining candidate is scored without any FE analysis, the B highest-scoring candidates are analysed with PyNite and added to the training set. This is pool-based active learning [20] in the spirit of adaptive surrogate methods in structural reliability [2, 3]. Four acquisition rules are compared: random sampling, the ensemble variance (mean normalised variance over all outputs), the equilibrium residual of the raw prediction, and a diversity-aware variant of the latter. Selecting the B largest scores can pick many near-identical frames from one corner of the design space, so the diversity-aware rule shortlists the $3B$ candidates with the largest residuals, groups them into B clusters by k-means on standardised frame descriptors (the mean and maximum of the node features) and takes the highest-residual frame of each cluster, in the spirit of diverse mini-batch active learning [38, 39]. Each rule is repeated with five random initial sets (seeds 0–4). The candidates are labelled in advance for convenience, but a label is revealed to the learner only when the strategy requests that analysis, and the recorded PyNite time of each requested analysis is charged to the budget.

FE referral at deployment. For a batch of new frames, all are first predicted and projected. A frame is sent to PyNite, and its surrogate result discarded, if it fails the domain check or if its conformal error bound exceeds the tolerance (Section 3.8); the remaining predictions are accepted. To describe how well the spread ranks the errors independently of any threshold, risk–coverage curves [40] are also reported, in which the fraction of frames with the largest spread referred to FE is varied.

5. Baselines and evaluation protocol

Baselines. Five baselines represent conventional regression and widely used machine learning: ridge regression [41] on the features (linear) and on all their pairwise products (quadratic), random forests [42] (200 trees), histogram gradient boosting [43] (600 iterations, one model per output) and a three-layer

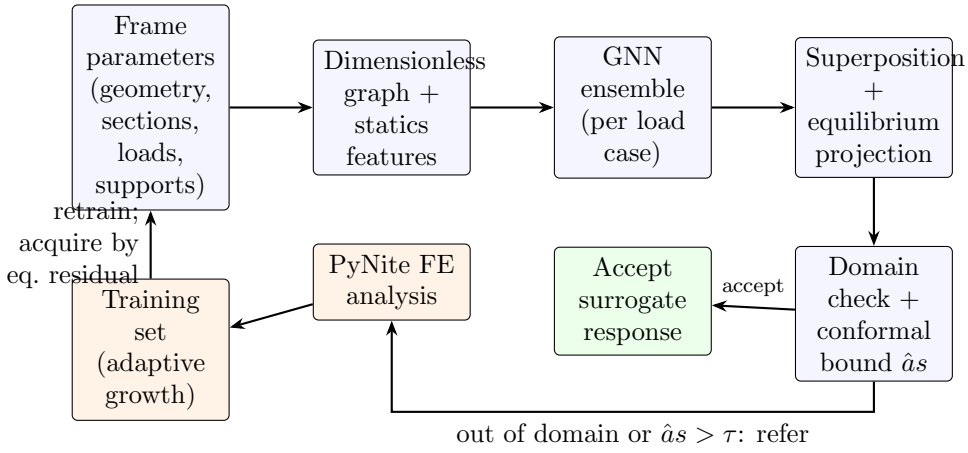


Figure 2: Proposed workflow. Exact mechanics (superposition, boundary conditions, static equilibrium) is imposed around a learned graph ensemble. During training, new FE analyses are chosen by the equilibrium residual of the raw prediction; at deployment, a domain check and a conformal bound on the ensemble spread decide which frames are referred to PyNite.

perceptron (256 units per layer, early stopping), all from scikit-learn [44]. Because fixed-length regressors cannot ingest graphs of varying size, they predict per node (displacements, from the 25 node features) and per member (basic forces, from the 10 member features concatenated with the features of both end nodes). They receive the same dimensionless, superposition-split targets and the same hard boundary conditions as the graph network, so that the comparison isolates the learning model and the constraint layer rather than the preprocessing. The quadratic ridge model is fitted on a random subset of 30,000 rows to keep the design matrix in memory. All models are trained on the same first 3,000 frames of the ID pool.

Graph-learning baselines. Five graph models isolate the design choices of the proposed network. All are trained on the same 3,000 frames for 150 epochs with three random seeds each. (i) A *conventional GNN*: the same architecture, but with physical inputs (coordinates, support flags, nodal loads, member length, $\log_{10} E$, $\log_{10} I$, $\log_{10} A$, line load) and physical targets for the combined load case, standardised over the training set; no statics features, no load-case split and no dimensional scaling. (ii) A *global-pooling* variant of the proposed network, in which the graph-wide mean of the node states is appended to every node update, giving every node frame-level context at

every step. (iii) A *StructGNN-style* network following Chou et al. [22]: one pseudo node per storey linked to all joints of that floor and to the pseudo node below, linear encoders to width 256, linear messages with mean aggregation in both edge directions, $n_s + 1$ propagation steps for a frame with n_s storeys (mini-batches grouped by n_s), decoders with one ReLU hidden layer and a mean-absolute-error loss. To allow a fair comparison it receives the same dimensionless features and targets as the proposed network. Two deviations from the original are needed: the weights are shared across propagation steps, because the unseen-geometry frames need up to nine steps whereas training frames need at most six, and a residual connection is added to train stably with the learning-rate schedule used for all models (the original used 1,000 epochs with batch size one, which was not affordable for all seeds here). (iv) A *displacement-only* network: the proposed architecture trained on the displacements only, with member forces recovered from the predicted displacements through the element stiffness relations and the fixed-end forces. (v) A *Leng-style* network following Leng et al. [26]: as (iv), with the normalised free-DOF stiffness residual $\|\mathbf{K}_{ff}\hat{\mathbf{u}}_f - \mathbf{f}_f\|^2/\|\mathbf{f}_f\|^2$ added to the loss with weight 10^{-2}, chosen from $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$ on a short pilot run because larger weights let the poorly conditioned axial rows dominate. These are re-implementations from the published descriptions, not the authors' code. The soft-equilibrium-penalty ablation, a depth ablation (4, 8, 12, 16 message-passing steps) and a feature ablation are trained with three seeds as well. The feature ablation separates the three ingredients of the input and output representation (Table 11): A is the conventional GNN (i); B uses dimensionless scaling and separate load cases but removes the ten statics-derived features (storey shear, overturning moment, tributary gravity load, stiffness ratios, shear-building drift and displacement; Table A.21); C keeps the statics features but replaces the load-case split by a single combined load scale; D is the proposed network. A and B still differ in more than one respect (physical versus dimensionless inputs and targets, and combined versus separate cases), so their difference is attributed to scaling and load-case separation jointly.

Metrics. All errors are computed per frame in physical units against PyNite and summarised over a test set by the median and the 95th percentile (p95). With $\hat{\cdot}$ the surrogate and $\|\cdot\|$ the Euclidean norm over all entries of a frame, the field errors are relative L_2 errors, $e_a = \|\hat{\mathbf{a}} - \mathbf{a}\|/\|\mathbf{a}\|$, for the horizontal displacements (e_{u_x}), vertical displacements (e_{u_y}) and rotations (e_θ) of

all nodes, the end moments (e_M), axial forces (e_N) and end shears (e_V) of all members, and the support reactions (e_R). The peak-response errors are $|\max \hat{a} - \max a| / \max a$ for the largest inter-storey drift ratio (e_{drift}), the largest beam mid-span deflection relative to the chord (e_{defl}; Hermite interpolation of the end rotations plus the fixed-end deflection $wL^4/(384EI)$), the largest absolute end moment ($e_{M,\max}$) and the largest absolute axial force ($e_{N,\max}$). The governing-member agreement is the fraction of frames in which the member with the largest end moment is predicted as the largest (top-1) or among the three largest (top-3). The equilibrium residual is $\|\mathbf{r}\|/\|\mathbf{P}_f - \mathbf{p}_f^0\|$ with $\mathbf{r}$ the nodal out-of-balance force at the free DOFs, moment rows divided by H; the compatibility residual is Eq. (6).

Statistics. The proposed ensemble is trained five times independently (five ensembles of five networks, 25 networks in total, with disjoint seeds). For each metric the per-run median and p95 are reported as mean ± standard deviation over the five runs, with a 95% confidence interval from the t distribution with four degrees of freedom. Within one run, 95% confidence intervals of the median and p95 over test frames are obtained by the percentile bootstrap [45] with 2,000 resamples, and differences between two variants evaluated on the same frames (e.g. raw versus projected forces) by a paired bootstrap. Graph baselines are reported as mean ± standard deviation over three seeds; the tabular baselines, which are deterministic for a given training set apart from the random forest and boosting seeds, are reported with frame-bootstrap intervals of the median (Table D.22 in Appendix D). All runs share one training set, one validation set and one set of test frames, so the across-run intervals quantify the variability due to network initialisation and batch order and the bootstrap intervals the sampling of test frames; neither covers the variability that a different training sample or a different design space would cause.

Computing environment. All experiments ran on a cloud virtual machine with four cores of an Intel Xeon processor at 2.1 GHz, 15 GB of memory and no GPU, under Linux with Python 3.13, PyNite 3.2.0, NumPy 2.5, SciPy 1.18, scikit-learn 1.9, PyTorch 2.15 and OpenSeesPy 3.8. Networks were trained on the CPU with one thread each, four in parallel; timing measurements were taken separately on the otherwise idle machine (Section 6.10).

6. Results

All numbers in this section are produced by the scripts of the reproducibility package. Unless stated otherwise, errors are medians over the frames of a test set (600 frames for ID, OOD-G and OOD-L, 300 for DES), the proposed surrogate is the five-network ensemble with variance-weighted projection, and single numbers refer to the first of five independent training runs (ensemble 0); the variation over the five runs is reported in Section 6.3.

6.1. Dataset

The 5,000-frame pool contains frames with 3 to 45 members (18 on average), the OOD-G set frames with 11 to 104 members (56 on average) and the DES set frames with 3 to 45 members (18 on average); Fig. 3 shows one frame of each set. A PyNite analysis of one frame (both load cases, including model construction and result extraction) took 37.5 ms on average for the ID frames and 104.7 ms for the larger OOD-G frames (Section 6.10). Because sections and loads of the ID, OOD-G and OOD-L frames are sampled independently and are not sized for serviceability, the median largest inter-storey drift ratio is 0.46% (ID) and 0.79% (OOD-G), and 15% and 25% of these frames exceed 2%. The design-informed frames are stiffer, with a median largest drift ratio of 0.18%; 29% of them exceed $h/400$ because the column sizing uses a simplified shear-building estimate. Since all models predict dimensionless responses that are exactly invariant to the load intensities and to E (Section 3.2), the relative errors reported below do not depend on the load magnitudes.

6.2. Accuracy on seen and unseen frames

Tables 5 and 6 compare the proposed surrogate with the five tabular baselines, and Fig. 4 shows the full error distributions. On the in-distribution test set the proposed surrogate reaches median errors of 4.5% for the horizontal displacement field, 4.9% for the governing drift ratio, 2.4% for the end-moment field and 0.6% for the axial forces. The best baseline, gradient boosting, has 29.0%, 30.2%, 10.0% and 3.2%, so the graph surrogate is about four times more accurate for the member forces and six times more accurate for the displacements, although the baselines receive the same mechanics-informed features, targets and boundary conditions. The gap is largest for displacements, which depend on the stiffness of the whole frame and not

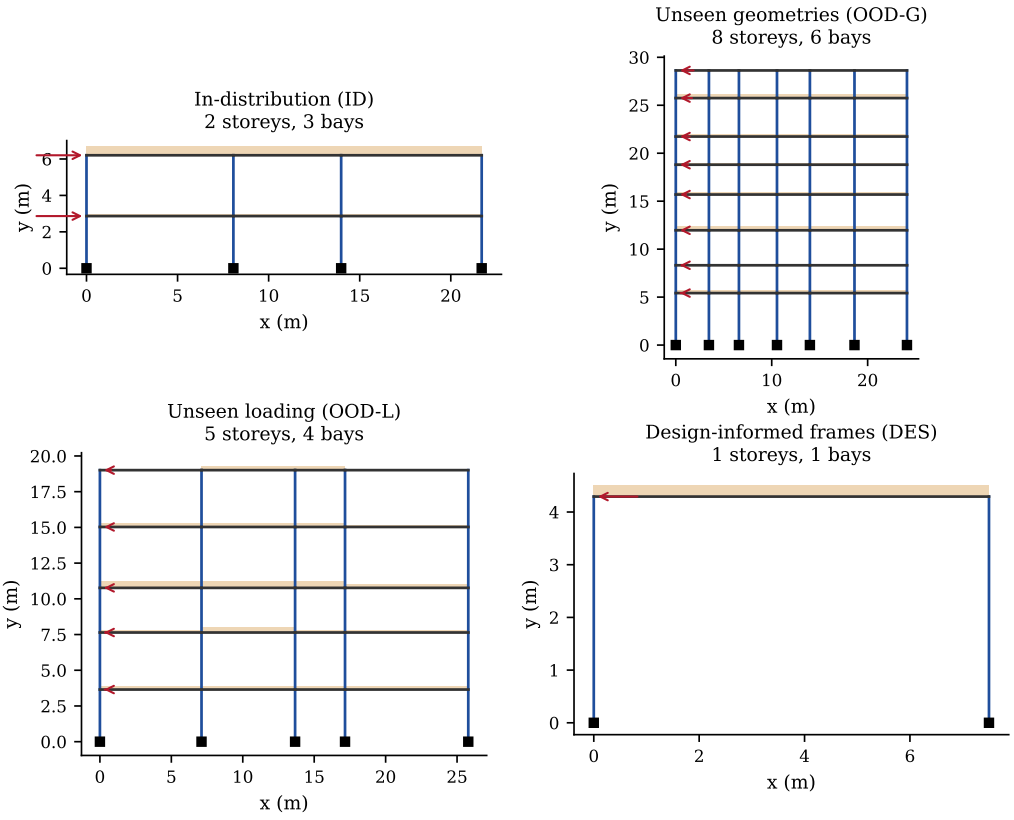


Figure 3: Example frames from the four test sets. Blue: columns; black: beams; shaded: gravity line loads (scaled); red arrows: lateral floor forces; squares and triangles: fixed and pinned bases.

Table 5: Prediction errors on the in-distribution and design-informed test sets (medians over the frames, in %, except the 95th percentile (p95) of e_M; last column: median relative nodal out-of-balance force). Bold: best value per column and set. The last row of each block gives the proposed surrogate as mean ± standard deviation of the per-run values over five independent training runs.

Model	e_{u_x}	e_{drift}	e_M	e_M (p95)	e_N	Out-of-balance
In-distribution (ID) (600 frames)						
Linear ridge	53.5	56.0	38.1	61.5	20.6	4.3×10^{-1}
Quadratic ridge	51.0	66.0	18.4	40.4	5.7	1.0×10^{-1}
Random forest	33.6	40.5	16.9	31.1	6.9	1.4×10^{-1}
Gradient boosting	29.0	30.2	10.0	22.3	3.2	7.4×10^{-2}
MLP	39.5	59.9	10.8	25.4	4.9	1.2×10^{-1}
GNN ensemble, raw forces	**4.5**	**4.9**	2.5	**5.9**	1.8	4.2×10^{-2}
Proposed (ensemble + var.-weighted proj.)	**4.5**	**4.9**	**2.4**	6.5	**0.6**	$< 10^{-12}$
Proposed, 5 independent runs (mean ± s.d.)	4.5 ± 0.2	4.6 ± 0.2	2.4 ± 0.0	6.6 ± 0.1	0.6 ± 0.0	–
Design-informed frames (DES) (300 frames)						
Linear ridge	118.3	138.4	35.5	60.6	15.1	3.7×10^{-1}
Quadratic ridge	93.3	150.3	14.7	25.7	4.7	7.8×10^{-2}
Random forest	42.9	54.0	13.5	20.7	4.9	9.6×10^{-2}
Gradient boosting	37.3	55.3	8.7	15.1	2.4	5.8×10^{-2}
MLP	45.2	66.9	8.7	12.5	4.0	10.0×10^{-2}
GNN ensemble, raw forces	**4.4**	**3.0**	2.2	4.6	1.4	3.3×10^{-2}
Proposed (ensemble + var.-weighted proj.)	**4.4**	**3.0**	**2.0**	**3.4**	**0.3**	$< 10^{-12}$
Proposed, 5 independent runs (mean ± s.d.)	4.5 ± 0.1	3.7 ± 0.4	2.0 ± 0.1	3.6 ± 0.2	0.3 ± 0.0	–

Table 6: As Table 5, for the unseen-geometry and unseen-loading test sets.

Model	e_{u_x}	e_{drift}	e_M	e_M (p95)	e_N	Out-of-balance
Unseen geometries (OOD-G) (600 frames)						
Linear ridge	61.4	72.3	82.2	192.6	36.2	1.2×10^{0}
Quadratic ridge	66.6	141.2	44.6	116.0	11.3	4.0×10^{-1}
Random forest	41.9	92.6	32.5	55.9	15.9	3.4×10^{-1}
Gradient boosting	43.1	118.3	22.5	41.1	6.2	1.8×10^{-1}
MLP	48.5	143.3	22.1	40.4	10.0	3.4×10^{-1}
GNN ensemble, raw forces	**11.4**	**13.7**	8.3	22.7	5.3	1.6×10^{-1}
Proposed (ensemble + var.-weighted proj.)	**11.4**	**13.7**	**7.5**	**22.6**	**1.5**	$< 10^{-12}$
Proposed, 5 independent runs (mean ± s.d.)	12.1 ± 0.5	13.9 ± 0.4	7.6 ± 0.2	22.5 ± 2.4	1.5 ± 0.1	–
Unseen loading (OOD-L) (600 frames)						
Linear ridge	52.8	57.5	44.3	83.6	26.2	5.2×10^{-1}
Quadratic ridge	53.2	75.8	26.7	49.0	9.4	2.1×10^{-1}
Random forest	33.8	42.9	24.2	38.2	12.0	2.3×10^{-1}
Gradient boosting	28.4	32.3	17.3	30.8	4.8	1.2×10^{-1}
MLP	38.0	50.1	16.8	31.4	6.4	1.5×10^{-1}
GNN ensemble, raw forces	**5.6**	**5.7**	9.1	21.5	3.5	6.7×10^{-2}
Proposed (ensemble + var.-weighted proj.)	**5.6**	**5.7**	**8.7**	**21.4**	**1.6**	$< 10^{-12}$
Proposed, 5 independent runs (mean ± s.d.)	5.9 ± 0.1	5.7 ± 0.3	8.8 ± 0.3	21.6 ± 0.5	1.5 ± 0.0	–

Table 7: Extended response metrics of the proposed surrogate: median and 95th percentile (p95) of the per-frame errors (%), mean ± standard deviation over five independent training runs. Governing member: share of frames in which the member with the largest end moment is predicted as the largest (top-1) or among the three largest (top-3).

	ID		OOD-G		OOD-L		DES	
Metric	median	p95	median	p95	median	p95	median	p95
e_{u_x}	4.5 ± 0.2	15.5 ± 0.3	12.1 ± 0.5	38.4 ± 1.7	5.9 ± 0.1	18.1 ± 0.9	4.5 ± 0.1	12.5 ± 0.7
e_{u_y}	3.2 ± 0.1	11.0 ± 0.5	6.8 ± 0.2	18.4 ± 0.7	7.5 ± 0.2	18.2 ± 0.6	3.2 ± 0.2	7.0 ± 0.5
e_θ	4.2 ± 0.0	12.8 ± 0.4	10.1 ± 0.2	27.7 ± 2.4	10.1 ± 0.2	39.7 ± 0.6	4.3 ± 0.3	10.2 ± 0.7
e_{drift}	4.6 ± 0.2	20.4 ± 0.8	13.9 ± 0.4	97.3 ± 7.0	5.7 ± 0.3	24.6 ± 0.5	3.7 ± 0.4	19.7 ± 3.3
e_{defl}	0.6 ± 0.0	4.3 ± 0.4	0.9 ± 0.1	8.9 ± 0.8	2.1 ± 0.1	16.9 ± 0.5	0.6 ± 0.0	2.8 ± 0.3
e_M	2.4 ± 0.0	6.6 ± 0.1	7.6 ± 0.2	22.5 ± 2.4	8.8 ± 0.3	21.6 ± 0.5	2.0 ± 0.1	3.6 ± 0.2
e_N	0.6 ± 0.0	2.4 ± 0.1	1.5 ± 0.1	4.2 ± 0.1	1.5 ± 0.0	4.5 ± 0.1	0.3 ± 0.0	0.9 ± 0.1
e_V	1.2 ± 0.0	3.6 ± 0.1	3.7 ± 0.2	8.3 ± 0.3	4.1 ± 0.1	10.4 ± 0.2	0.8 ± 0.0	1.7 ± 0.1
e_R	0.6 ± 0.0	2.3 ± 0.1	1.2 ± 0.0	3.6 ± 0.1	1.3 ± 0.0	5.4 ± 0.1	0.3 ± 0.0	0.7 ± 0.1
$e_{M,\max}$	0.8 ± 0.0	3.7 ± 0.1	2.2 ± 0.1	9.6 ± 0.8	2.6 ± 0.2	14.4 ± 0.3	0.8 ± 0.1	2.5 ± 0.3
$e_{N,\max}$	0.2 ± 0.0	1.2 ± 0.1	0.4 ± 0.0	2.3 ± 0.1	0.4 ± 0.0	2.6 ± 0.2	0.1 ± 0.0	0.4 ± 0.0
Governing member, top-1 (%)	94.1 ± 0.7		86.5 ± 0.9		85.6 ± 0.3		93.5 ± 1.3	
Governing member, top-3 (%)	99.9 ± 0.1		97.7 ± 0.4		99.5 ± 0.3		99.9 ± 0.3	

only on local quantities. The design-informed frames, whose parameter correlations were never seen in training, are predicted as accurately as the ID frames: median errors of 4.4% (u_x), 3.0% (drift) and 2.0% (moments), with a 95th percentile of the moment error of only 3.4%, against 8.7% (moments) and 37% (u_x) for the best baselines.

All models lose accuracy on the unseen geometries and load patterns, but the ranking is unchanged. For unseen geometries (up to eight storeys and six bays, against at most five and four in training) the proposed surrogate has a median moment error of 7.5% and a median displacement error of 11.4%, against 22.5% and 43.1% for gradient boosting; the governing-drift error is 13.7%, whereas the tabular baselines have errors of 72–143%. For unseen load patterns the median moment error is 8.7% (gradient boosting 17.3%, MLP 16.8%) and the displacement error 5.6%. The tails are heavy: the 95th percentile of the moment error is 22.6% for unseen geometries and 21.4% for unseen loading, which is not acceptable for design without further checks and motivates the referral rule of Section 6.8. Figure 5 shows parity plots for the roof displacement and the largest end moment, and Fig. 6 the end moments of a typical unseen-geometry frame.

Table 7 extends the evaluation to the quantities an engineer checks. Vertical displacements and rotations are predicted with median errors of 3.2% and 4.2% on ID frames. The largest beam mid-span deflection, the largest end moment and the largest axial force, which govern serviceability and member

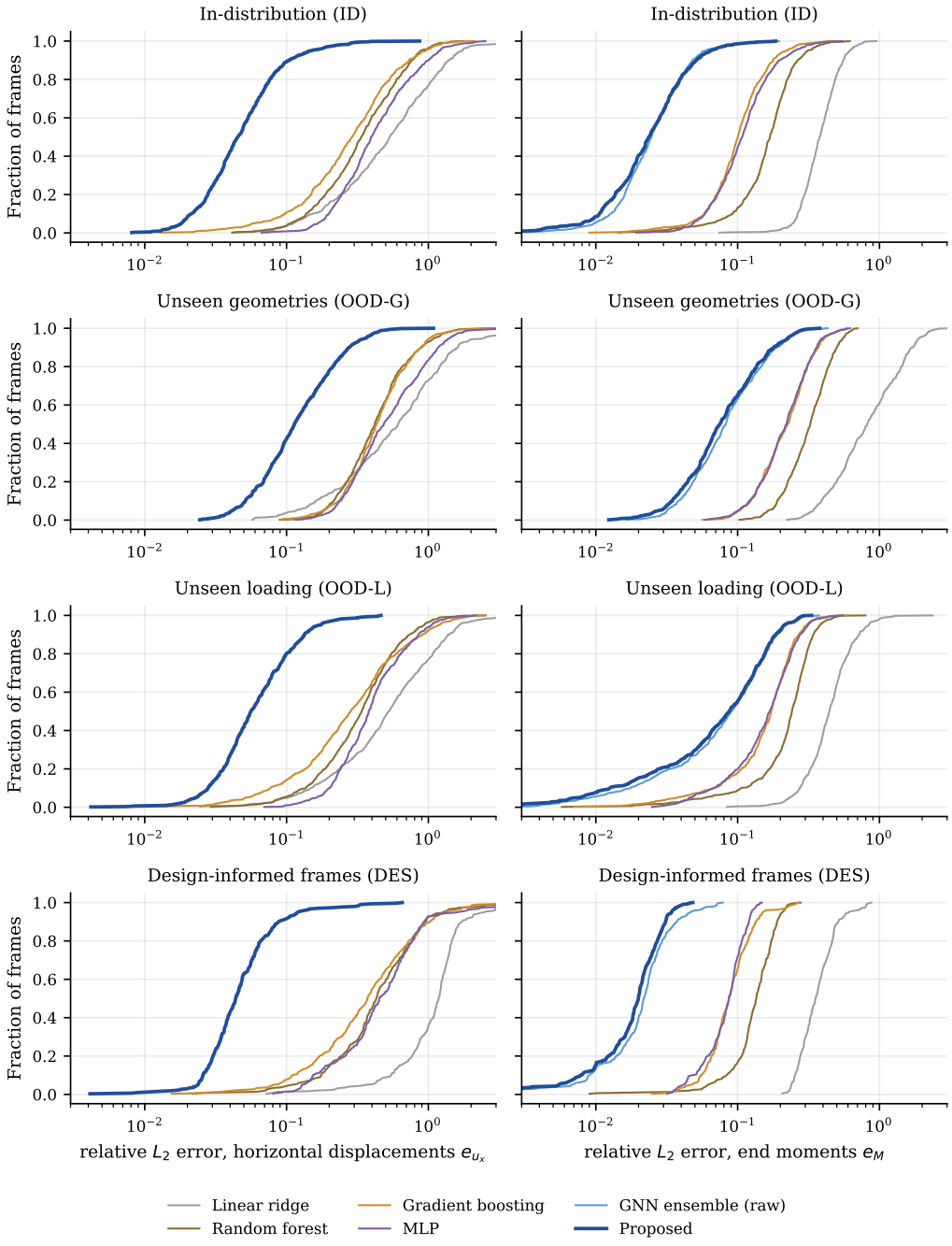


Figure 4: Cumulative distributions of the per-frame relative errors of the horizontal displacement field (left) and of the end-moment field (right) for the four test sets.

Table 8: Uncertainty of the headline results (moment error e_M, %). Bootstrap: 95% percentile-bootstrap interval over test frames for ensemble 0 (2,000 resamples). Across runs: mean and 95% t-interval of the per-run median over five independent ensembles. Raw − projected: reduction of the median e_M by the variance-weighted projection, paired bootstrap for ensemble 0 and t-interval over the five runs.

Set	Median (bootstrap)	p95 (bootstrap)	Median (across runs)	Raw − proj. (bootstrap)	Raw − proj. (across runs)
ID	2.36 [2.24, 2.52]	6.5 [5.8, 7.2]	2.37 [2.34, 2.40]	0.10 [-0.01, 0.20]	0.10 [0.05, 0.14]
OOD-G	7.51 [6.95, 8.13]	22.6 [20.6, 24.6]	7.65 [7.35, 7.95]	0.75 [0.34, 1.00]	0.53 [0.33, 0.72]
OOD-L	8.73 [7.91, 9.87]	21.4 [20.4, 25.0]	8.79 [8.39, 9.18]	0.36 [-0.04, 0.77]	0.27 [-0.05, 0.60]
DES	2.00 [1.87, 2.06]	3.4 [3.1, 3.6]	1.99 [1.89, 2.10]	0.18 [0.13, 0.27]	0.17 [0.08, 0.26]

design, are within 0.6%, 0.8% and 0.2% of PyNite at the median, and the support reactions within 0.6%; the 95th percentiles of these peak quantities stay below 4.5% on the ID and DES sets. End shears follow the moments (median 1.2%). The member with the largest end moment is identified correctly in about 94% of the ID frames and is among the three predicted largest in almost all frames. On the unseen sets the peak quantities degrade much less than the full fields (largest moment 2.2% and 2.5% at the median for OOD-G and OOD-L), whereas the rotations and the governing drift ratio show the largest tails (95th percentiles of 28–40% for the rotations and up to 97% for the governing drift on OOD-G), so serviceability checks on unseen frames are the least reliable use of the surrogate.

6.3. Statistical robustness

The five independent training runs (25 networks) give almost the same errors (Tables 5–8): the per-run median moment error is 2.37 ± 0.03% (ID), 7.65 ± 0.24% (OOD-G), 8.79 ± 0.32% (OOD-L) and 1.99 ± 0.08% (DES), and the median displacement error 4.45 ± 0.15%, 12.1 ± 0.5%, 5.85 ± 0.15% and 4.45 ± 0.12%. The variation between runs is much smaller than the sampling uncertainty of a single run over the test frames: the 95% bootstrap interval of the ID median moment error of ensemble 0 is [2.24, 2.52]%, and that of the 95th percentile [5.8, 7.2]%. Every difference between the proposed surrogate and the tabular baselines in Tables 5–6 is far larger than these intervals.

The effect of the projection on the median moment error is small and must be read with its intervals. On the ID set it lowers the median by 0.10 percentage points (2.46%→2.36% for ensemble 0). The reduction has the same sign in all five runs (95% t-interval [0.05, 0.14] points), but the paired bootstrap interval for a single run, [−0.01, 0.20] points, includes zero. The reduction is larger and significant in both senses on unseen geometries (0.53

points across runs, [0.33, 0.72]) and on the DES set (0.17, [0.08, 0.26]); on unseen load patterns the across-run interval includes zero. The projection should therefore be credited with exact equilibrium and a large reduction of the axial-force errors (Section 6.5), and with a modest, consistent reduction of the moment errors, not with a decisive gain in moment accuracy.

6.4. Graph-learning baselines, ablations and message-passing depth

Table 9 compares single networks, so that the architecture is separated from the ensemble and the projection. Four findings stand out.

Dimensionless scaling and the load-case split, not the statics features, carry most of the gain. The feature ablation (Table 11) separates the three ingredients. Going from the conventional GNN (A) to dimensionless, load-case-separated inputs and targets without any statics features (B) halves the displacement errors (14.3%→6.9% ID, 34.2%→14.9% OOD-G, 27.9%→8.4% DES) and reduces the moment errors by 1.5 points (ID) and 5.1 points (OOD-G), but only by 0.4 points on DES frames, while on unseen load patterns the moment error rises from 8.4% to 9.3%; because A and B differ in several respects at once, this gain is attributed to scaling and load-case separation jointly. Adding the statics features (B→D) changes the errors much less: on ID, OOD-G and DES frames the displacement and moment errors fall by 0.2–1.2 points, on unseen load patterns the displacement error falls from 12.0% to 8.1%, but the moment error rises from 9.3% to 10.0%. Removing the load-case split while keeping the statics features (C) roughly doubles the displacement errors (12.5% ID, 26.0% OOD-G, 34.9% DES) and raises the moment errors by up to 3.5 points, confirming the scaling argument of Section 3.2: in the combined case the lateral sway is masked by the much larger gravity scale. The improvement over a conventional GNN is therefore due mainly to the combined use of dimensionless scaling and load-case separation; the statics features add a smaller, mainly displacement-related benefit, which is clearest under unseen load patterns.

The StructGNN-style network is a strong competitor. With the same inputs and targets it reaches displacement errors similar to or lower than the proposed network (6.3% ID, 12.3% OOD-G, 7.1% OOD-L) and slightly higher moment errors on ID, OOD-G and DES frames (4.2%, 11.9% and 3.9% against 3.7%, 11.3% and 3.3%); on unseen load patterns it is marginally better (9.7% against 10.0%). Its storey pseudo nodes and height-dependent depth carry global information quickly, which helps the displacements, at seven times the training cost (77 against 11 minutes per network on one

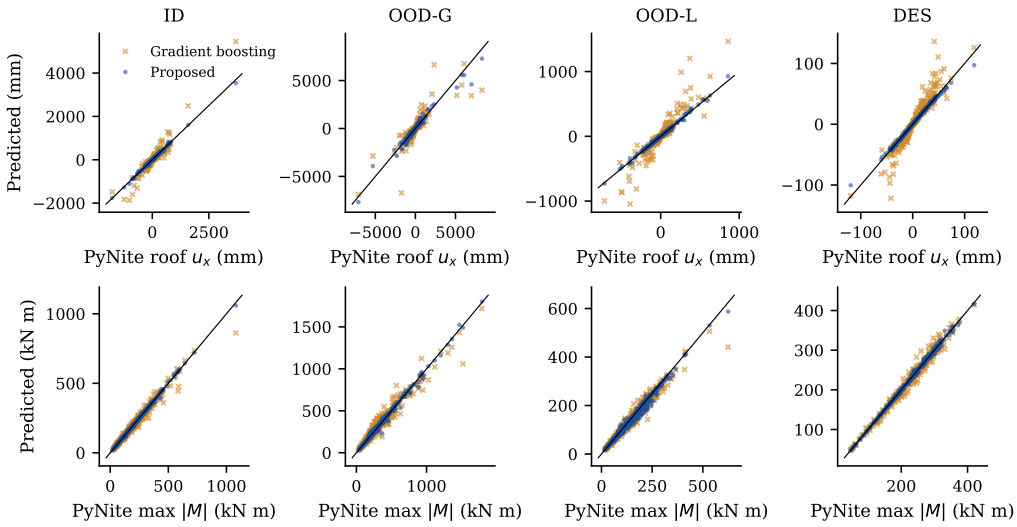


Figure 5: Predicted versus PyNite roof displacement (top; mean of the roof nodes) and largest absolute end moment (bottom) for the proposed surrogate and the best tabular baseline.

Table 9: Single graph networks (no ensembling, no projection): median e_{u_x} and e_M (%), mean ± standard deviation over three training seeds. All models are trained on the same 3,000 frames for 150 epochs.

	ID		OOD-G		OOD-L		DES	
Model	e_{u_x}	e_M	e_{u_x}	e_M	e_{u_x}	e_M	e_{u_x}	e_M
Proposed network, single (8 steps)	6.7 ± 0.1	3.7 ± 0.1	14.1 ± 0.7	11.3 ± 0.5	8.1 ± 0.1	10.0 ± 0.1	7.2 ± 0.3	3.3 ± 0.0
Conventional GNN (physical inputs/targets)	14.3 ± 0.4	5.4 ± 0.1	34.2 ± 2.2	16.9 ± 0.6	22.2 ± 0.9	8.4 ± 0.1	27.9 ± 2.8	4.3 ± 0.1
Proposed + global pooling	6.5 ± 0.2	3.7 ± 0.0	11.1 ± 0.7	10.9 ± 0.7	8.3 ± 0.4	10.3 ± 0.2	6.5 ± 0.7	3.4 ± 0.1
StructGNN-style	6.3 ± 0.1	4.2 ± 0.1	12.3 ± 0.9	11.9 ± 0.2	7.1 ± 0.3	9.7 ± 0.1	7.4 ± 0.2	3.9 ± 0.0
Displacement-only + force recovery	7.4 ± 0.3	20.1 ± 0.4	14.7 ± 1.2	74.7 ± 5.8	8.7 ± 0.2	26.0 ± 0.1	7.3 ± 0.5	6.0 ± 0.3
Leng-style, residual weight 10^{-2}	48.2 ± 3.7	46.0 ± 2.8	59.6 ± 3.8	75.9 ± 2.6	54.2 ± 4.3	48.8 ± 2.1	46.4 ± 3.2	14.8 ± 1.7
Leng-style, residual weight 10^{-4}	14.3 ± 0.5	29.7 ± 1.2	21.3 ± 0.3	74.0 ± 1.2	13.2 ± 0.5	30.5 ± 1.8	19.2 ± 3.5	9.5 ± 0.2
Proposed + soft equilibrium penalty	6.4 ± 0.1	3.6 ± 0.0	13.7 ± 0.7	11.3 ± 0.7	8.1 ± 0.2	9.9 ± 0.2	7.0 ± 0.3	3.4 ± 0.2
Proposed, no load-case split	12.5 ± 0.5	5.5 ± 0.1	26.0 ± 1.5	14.8 ± 0.2	15.5 ± 0.5	10.0 ± 0.1	34.9 ± 4.0	4.2 ± 0.1
Proposed, no statics features	6.9 ± 0.2	3.9 ± 0.1	14.9 ± 0.5	11.8 ± 0.3	12.0 ± 0.5	9.3 ± 0.4	8.4 ± 0.8	3.9 ± 0.1
Message-passing depth (proposed network, single)								
Depth 4	9.1 ± 0.3	4.2 ± 0.1	20.1 ± 0.8	12.5 ± 0.2	10.8 ± 0.2	10.5 ± 0.2	10.3 ± 0.9	4.0 ± 0.1
Depth 12	6.3 ± 0.0	3.6 ± 0.1	13.1 ± 1.0	11.2 ± 0.1	7.7 ± 0.3	9.9 ± 0.3	6.3 ± 0.3	3.4 ± 0.1
Depth 16	5.8 ± 0.0	3.6 ± 0.1	11.8 ± 0.6	11.2 ± 0.3	7.5 ± 0.6	10.1 ± 0.3	6.2 ± 0.4	3.4 ± 0.1

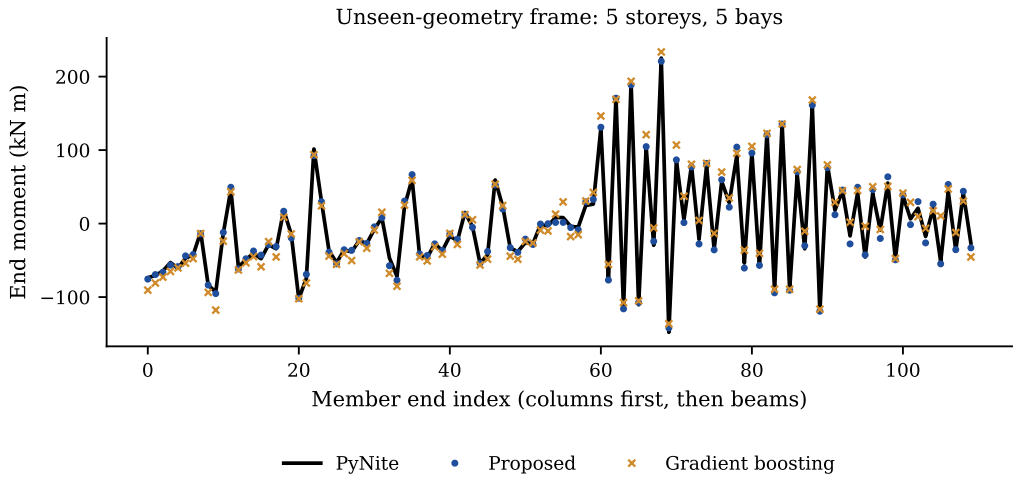


Figure 6: End moments of all members of an unseen-geometry frame (the OOD-G frame with the median number of members): PyNite, proposed surrogate and gradient boosting.

thread). The proposed network with global pooling or with 16 message-passing steps gains in the same way (OOD-G displacement error 11.1% and 11.8% against 14.1%) at little or no cost in moment accuracy. The accuracy of a single network is therefore not what distinguishes the proposed framework; the distinguishing elements are the exact constraints, the uncertainty estimate and its use, which are evaluated in the following sections and could equally be wrapped around a StructGNN-style network.

Depth mainly affects the displacements (Fig. 7). Going from 4 to 8 message-passing steps reduces the median displacement error from 9.1% to 6.7% (ID) and from 20.1% to 14.1% (OOD-G); 12 and 16 steps reduce it further to 5.8% and 11.8% (16 steps), whereas the moment errors improve from 4 to 8 steps (4.2%→3.7% ID) and change by at most 0.1 points beyond 8 steps. Displacements depend on the stiffness of the whole frame and benefit from information travelling across the full graph diameter (up to 14 members in the OOD-G frames, against 9 in training), while end moments are governed mainly by the neighbourhood of the member and by the statics features. Eight steps were kept for the main model because they halve the training cost of 16 steps and the ensemble recovers most of the difference (ensemble displacement error 4.5% ID, 11.4% OOD-G).

Displacement-based force recovery is much less accurate than predicting

Table 10: Equilibrium and compatibility of the graph baselines (single networks, mean of three seeds): median relative out-of-balance force of the raw forces, median e_M (%) after Euclidean projection of the forces, median moment compatibility residual $r_{\mathrm{compat},M}$ (%), and training time of one network on one CPU thread (min).

	ID			OOD-G			
Model	Eq. res.	e_M proj.	$r_{\mathrm{compat},M}$	Eq. res.	e_M proj.	$r_{\mathrm{compat},M}$	Train (min)
Proposed network, single (8 steps)	6.7×10^{-2}	5.6	18.0	2.3×10^{-1}	17.9	56.5	11
Conventional GNN (physical inputs/targets)	8.6×10^{-2}	4.4	36.5	2.3×10^{-1}	12.7	72.6	11
Proposed + global pooling	6.7×10^{-2}	5.5	19.0	2.3×10^{-1}	17.1	53.5	13
StructGNN-style	6.1×10^{-2}	5.3	18.2	2.2×10^{-1}	17.2	54.3	76
Displacement-only + force recovery	3.3×10^{0}	212.2	0.0	1.8×10^{1}	1094.2	0.0	11
Leng-style, residual weight 10^{-2}	4.9×10^{-1}	35.4	0.0	1.1×10^{0}	62.6	0.0	12
Leng-style, residual weight 10^{-4}	8.3×10^{-1}	46.5	0.0	2.9×10^{0}	147.4	0.0	12
Proposed + soft equilibrium penalty	6.2×10^{-2}	5.3	17.5	2.2×10^{-1}	16.8	55.9	13
Proposed, no load-case split	7.7×10^{-2}	7.4	32.7	2.4×10^{-1}	19.7	82.4	11
Proposed, no statics features	8.4×10^{-2}	6.5	19.3	3.1×10^{-1}	22.5	62.3	13

Table 11: Feature ablation (single networks, mean ± standard deviation of the median errors over three seeds, %).

				ID		OOD-G		OOD-L		DES	
Variant	Scaling	Statics feat.	Separate cases	e_{u_x}	e_M	e_{u_x}	e_M	e_{u_x}	e_M	e_{u_x}	e_M
A	No	No	No	14.3 ± 0.4	5.4 ± 0.1	34.2 ± 2.2	16.9 ± 0.6	22.2 ± 0.9	8.4 ± 0.1	27.9 ± 2.8	4.3 ± 0.1
B	Yes	No	Yes	6.9 ± 0.2	3.9 ± 0.1	14.9 ± 0.5	11.8 ± 0.3	12.0 ± 0.5	9.3 ± 0.4	8.4 ± 0.8	3.9 ± 0.1
C	Yes	Yes	No	12.5 ± 0.5	5.5 ± 0.1	26.0 ± 1.5	14.8 ± 0.2	15.5 ± 0.5	10.0 ± 0.1	34.9 ± 4.0	4.2 ± 0.1
D	Yes	Yes	Yes	6.7 ± 0.1	3.7 ± 0.1	14.1 ± 0.7	11.3 ± 0.5	8.1 ± 0.1	10.0 ± 0.1	7.2 ± 0.3	3.3 ± 0.0

forces directly. A network trained on displacements alone has displacement errors close to the proposed network (7.4% ID), but the forces recovered from its displacements have median moment errors of 20% (ID) and 75% (OOD-G), and their out-of-balance force exceeds the applied load (Table 10). Adding the stiffness residual to the loss in the Leng-style variant did not help in our implementation: at a weight of 10^{-2} the residual term dominates and the displacement errors rise to 48% (ID), and at 10^{-4} they are still 14%, with moment errors of 30% (ID). The free-DOF residual is dominated by the axial rows, whose stiffness exceeds the flexural stiffness by orders of magnitude, so penalising it pulls the network towards axial compatibility at the expense of the flexural response. A carefully scaled residual (for example, row-scaled by the diagonal of $\mathbf{K}$) may perform better, and this result should not be read as a verdict on the original method, whose code we did not use. Finally, the soft equilibrium penalty neither improves accuracy nor materially reduces the out-of-balance force (6.2% against 6.7% on ID frames), which supports enforcing equilibrium exactly instead of through the loss.

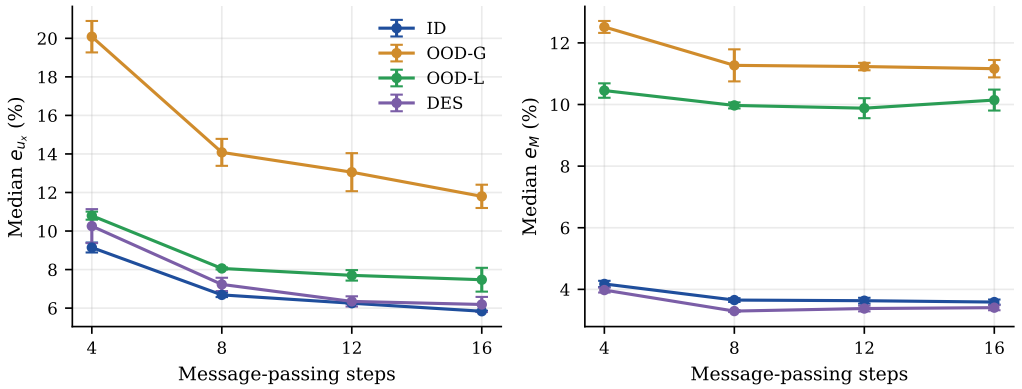


Figure 7: Effect of the number of message-passing steps on the median displacement (left) and moment (right) errors of single networks (mean ± standard deviation over three seeds).

6.5. Equilibrium and the projection layer

Without the projection, every model produces member forces that are not in equilibrium with the applied loads (Fig. 8). The median out-of-balance force is 4% of the load vector for the raw graph ensemble and 7–43% for the tabular baselines on in-distribution frames, rising to 16% and 18–120% for unseen geometries. After projection (Eq. 4) the residual is below 10^{-12} for every frame and every model, so the support reactions balance the loads and every nodal free body is in equilibrium.

The metric of the projection matters. With the variance-weighted metric the axial-force error decreases by a factor of 2–4.7 (1.8%→0.6% ID, 5.3%→1.5% OOD-G, 3.5%→1.6% OOD-L, 1.4%→0.3% DES; Table 13), the support-reaction errors by a factor of 2–3.5, and the moment errors decrease modestly (Section 6.3). With the Euclidean metric the projection degrades the moments (2.5%→4.0% ID, 8.3%→12.6% OOD-G) while still improving the axial forces. This is consistent with Proposition 1: the error can only decrease in the metric of the projection, and the Euclidean metric on dimensionless forces lets the correction flow into the moments, whose uncertainty it ignores.

Because the layer is model-agnostic, it was also applied to the tabular baselines (Table 12), which provide no per-output variance, so the Euclidean metric is used. The axial-force errors decrease for every baseline and test set. The moment errors decrease for gradient boosting and for the MLP on most sets, but increase for the linear ridge model and the random forest;

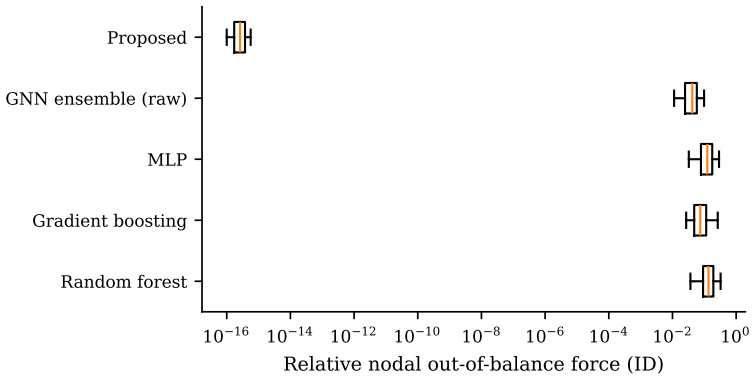


Figure 8: Relative nodal out-of-balance force of the predicted member forces on the in-distribution test set (box: quartiles; whiskers: 5th–95th percentiles). Only the projected surrogate is statically admissible.

Table 12: Effect of the Euclidean equilibrium projection on the tabular baselines: median moment error e_M and axial-force error e_N (%) before → after projection.

	ID		OOD-G		OOD-L		DES	
Model	e_M	e_N	e_M	e_N	e_M	e_N	e_M	e_N
Linear ridge	38.1 → 51.8	20.6 → 11.3	82.2 → 108.5	36.2 → 17.7	44.3 → 55.9	26.2 → 13.1	35.5 → 50.3	15.1 → 8.2
Random forest	16.9 → 18.2	6.9 → 4.2	32.5 → 33.7	15.9 → 6.2	24.2 → 26.4	12.0 → 5.8	13.5 → 16.2	4.9 → 3.2
Gradient boosting	10.0 → 9.1	3.2 → 2.2	22.5 → 21.4	6.2 → 4.2	17.3 → 16.5	4.8 → 3.4	8.7 → 8.5	2.4 → 1.5
MLP	10.8 → 10.5	4.9 → 2.5	22.1 → 27.0	10.0 → 5.0	16.8 → 16.5	6.4 → 3.5	8.7 → 10.4	4.0 → 2.0

in the conventional GNN, whose Euclidean metric acts on physical forces, the projection reduces the moment errors (5.4%→4.4% ID, 16.9%→12.7% OOD-G), whereas for the dimensionless single networks it increases them (Table 10). The projection therefore always delivers admissible forces and always improves the axial forces, while its effect on the moments depends on whether the metric reflects where the prediction errors actually are, which is what the ensemble variance provides.

6.6. Displacement–force compatibility

Table 13 answers the question whether the projection makes the force field more or less compatible with the predicted displacements. The forces implied by the predicted displacements differ substantially from the predicted forces: the median compatibility residual of the raw ensemble is 47% on the ID set and 87% on unseen geometries. Most of this comes from the axial forces, because a displacement error of a few per cent, multiplied by the

Table 13: Equilibrium and compatibility of the force fields of the proposed ensemble (ensemble 0): medians over the test frames (%, residuals as fractions). r_{compat}, $r_{\mathrm{compat},M}$ and $r_{\mathrm{compat},N}$: Eq. (6); eq. res.: relative nodal out-of-balance force. The displacements are the same in all rows.

Force field	e_M	e_N	e_V	Eq. res.	r_{compat}	$r_{\mathrm{compat},M}$	$r_{\mathrm{compat},N}$
In-distribution (ID)							
Raw ensemble forces	2.5	1.8	1.3	4.2×10^{-2}	47.2	11.2	47.7
Euclidean projection	4.0	1.1	2.1	$< 10^{-12}$	47.0	12.1	47.6
Variance-weighted projection	2.4	0.6	1.2	$< 10^{-12}$	47.0	11.4	47.7
Recovered from $\hat{\mathbf{u}}$	11.2	53.3	7.9	1.7×10^{0}	0.0	0.0	0.0
Unseen geometries (OOD-G)							
Raw ensemble forces	8.3	5.3	4.2	1.6×10^{-1}	87.5	39.7	87.6
Euclidean projection	12.6	2.5	6.3	$< 10^{-12}$	87.6	41.9	87.7
Variance-weighted projection	7.5	1.5	3.6	$< 10^{-12}$	87.6	40.2	87.7
Recovered from $\hat{\mathbf{u}}$	44.2	182.8	34.2	9.8×10^{0}	0.0	0.0	0.0
Unseen loading (OOD-L)							
Raw ensemble forces	9.1	3.5	5.0	6.7×10^{-2}	59.0	18.1	60.1
Euclidean projection	10.8	2.2	5.0	$< 10^{-12}$	59.0	20.2	60.1
Variance-weighted projection	8.7	1.6	4.2	$< 10^{-12}$	59.0	19.2	60.2
Recovered from $\hat{\mathbf{u}}$	18.6	76.3	12.1	2.1×10^{0}	0.0	0.0	0.0
Design-informed frames (DES)							
Raw ensemble forces	2.2	1.4	1.0	3.3×10^{-2}	15.1	3.9	15.2
Euclidean projection	3.9	0.7	1.7	$< 10^{-12}$	14.9	5.4	15.1
Variance-weighted projection	2.0	0.3	0.8	$< 10^{-12}$	14.9	4.0	15.0
Recovered from $\hat{\mathbf{u}}$	3.5	15.5	2.0	4.7×10^{-1}	0.0	0.0	0.0

large axial stiffness EA/L, produces axial forces that are far from the true ones; restricted to the axial forces the residual is 48% (ID), 88% (OOD-G), 60% (OOD-L) and 15% (DES), i.e. practically the full residual, whereas restricted to the end moments it is 11% (ID), 40% (OOD-G), 18% (OOD-L) and 4% (DES). The equilibrium projection hardly changes these numbers. With the variance-weighted metric, $r_{\mathrm{compat},M}$ increases in 68%, 67%, 74% and 69% of the ID, OOD-G, OOD-L and DES frames, but only by a median factor of 1.01–1.04; with the Euclidean metric the increase is larger (median factor 1.03–1.34) and affects 82–93% of the frames. The projection therefore slightly increases the incompatibility, as expected for a correction that acts on the forces alone, and the size of the increase follows the size of the force correction.

Recovering the forces from the predicted displacements, as in displacement-based surrogates [26], gives zero compatibility residual by construction, but much less accurate forces and large equilibrium

Table 14: Sensitivity of the variance-weighted projection to the regularisation of $\mathbf{\Sigma}$ (ensemble 0): median moment error e_M (%) and largest condition number of $\mathbf{C\Sigma C}^{\mathsf{T}}$ over the frames of each set.

	ID		OOD-G		OOD-L		DES	
$\mathbf{\Sigma}$	e_M	$\kappa_{\max}$	e_M	$\kappa_{\max}$	e_M	$\kappa_{\max}$	e_M	$\kappa_{\max}$
Euclidean ($\mathbf{\Sigma} = \mathbf{I}$)	4.05	4e3	12.57	2e4	10.84	4e3	3.91	2e3
$\max(\sigma^2, 1\,\bar{\sigma}^2)$	3.94	4e3	11.51	2e4	10.49	8e3	3.83	2e3
$\max(\sigma^2, 10^{-2}\bar{\sigma}^2)$	2.39	2e4	7.63	9e4	8.83	2e4	1.92	2e4
$\max(\sigma^2, 10^{-4}\bar{\sigma}^2)$	2.36	7e4	7.51	2e5	8.73	9e4	2.00	4e4
$\max(\sigma^2, 10^{-6}\bar{\sigma}^2)$	2.36	7e4	7.51	2e5	8.73	4e5	2.00	4e4
$\max(\sigma^2, 10^{-8}\bar{\sigma}^2)$	2.36	7e4	7.51	2e5	8.73	4e5	2.00	4e4
$\sigma^2 + 10^{-6}\bar{\sigma}^2$ (used)	2.36	7e4	7.51	2e5	8.73	3e5	2.00	4e4

violations: the median moment error rises from 2.4% to 11.2% (ID) and from 7.5% to 44.2% (OOD-G), the axial-force errors reach 53% and 183%, and the out-of-balance force exceeds the applied load (median residual 1.7 and 9.8). Member forces are second derivatives of the displacement field, so the differentiation amplifies the displacement errors. In this setting, predicting the forces directly and imposing equilibrium is far more accurate than imposing compatibility, which is why the proposed workflow enforces the former and only measures the latter. A field that satisfies both is the FE solution. Using the predicted displacements as the starting point of a Jacobi-preconditioned conjugate-gradient solve of the full stiffness equations did not reduce the number of iterations to a relative residual of 10^{-8} (median 27 iterations for ID frames with and without the warm start; 72 for OOD-G), because the initial residual is dominated by the stiff axial DOFs: at the median it is 1.4 times (ID) and 12 times (OOD-G) the norm of the load vector, i.e. larger than the residual of a zero initial guess. A compatibility-aware correction that would help in practice would have to treat the axial and flexural parts separately and is left for future work.

6.7. Sensitivity of the projection to the variance floor

Table 14 varies the floor of the variance-weighted metric. For floors between $10^{-8}\bar{\sigma}^2$ and $10^{-4}\bar{\sigma}^2$, and for the additive form of Eq. (5), the median moment errors are identical to two decimals on every set (2.36, 7.51, 8.73 and 2.00%), so the results do not depend on ε in this range. A floor of $10^{-2}\bar{\sigma}^2$ changes them by at most 0.12 percentage points, while a floor equal to the mean variance, which makes the metric nearly uniform, moves the

Table 15: Quality of the frame-level uncertainty indicators (ensemble 0, variance-weighted projection). ρ_s: Spearman rank correlation with the true moment error e_M; coverage: fraction of end moments within the ensemble mean $\pm 2\sigma$; ">10%": fraction of frames with $e_M > 10\%$; recall: share of those frames among the 20% with the largest spread; p95: 95th percentile of e_M over all frames and over the 80% with the smallest spread.

Set	ρ_s (spread)	ρ_s (eq. residual)	Coverage (%)	>10% (%)	Recall (%)	p95 all (%)	p95, 80% kept (%)
ID	0.81	0.57	92.3	1.5	78	6.5	4.8
OOD-G	0.92	0.59	92.2	34.8	56	22.6	15.0
OOD-L	0.66	0.43	66.9	44.8	37	21.4	20.5
DES	0.75	0.74	94.4	0.0	0	3.4	3.1

Table 16: Domain check (Steps A and B of Section 3.8): fraction of frames (%) flagged as outside the training domain. The Mahalanobis model is fitted on the training frames and the threshold is the conformal 99% quantile of the domain-calibration frames; neither set is shown. Cal.: conformal-calibration frames.

Descriptors	Cal.	ID	OOD-G	OOD-L	DES
Geometry descriptors	0.4	0.3	25.2	0.0	0.0
Load-pattern descriptors	0.9	1.5	9.5	66.7	7.3
Combined (used)	0.4	0.3	48.7	71.2	0.0

errors most of the way towards the Euclidean projection (3.9% ID, 11.5% OOD-G). The largest condition number of $\mathbf{C\Sigma C}^{\mathsf{T}}$ over all 2,100 test frames is 3.6×10^5 (median 4×10^3 on ID frames, 3.5×10^4 on OOD-G frames), which is harmless for the direct LU solve in double precision: the projected fields satisfy equilibrium to a relative residual below 10^{-12} in every frame.

6.8. Uncertainty, domain check and FE referral

The ensemble spread ranks the frames by their actual error well on in-distribution frames, on unseen geometries and on the design-informed frames (Spearman $\rho_s = 0.81$, 0.92 and 0.75; Table 15 and Fig. 9), and the $\pm 2\sigma$ intervals of the end moments contain 92%, 92% and 94% of the PyNite values, close to the nominal 95%. It is much weaker for unseen load patterns ($\rho_s = 0.66$, coverage 67%): the networks agree with each other on these frames while being wrong together, the known failure mode of ensembles under distribution shift [19]. The label-free equilibrium residual of the raw prediction is a weaker indicator ($\rho_s = 0.43$–0.74), which is why it is not used for referral.

The domain check (Table 16) flags 0.3% of the ID test frames and 0.4% of the conformal-calibration frames, within the nominal 1% for in-distribution

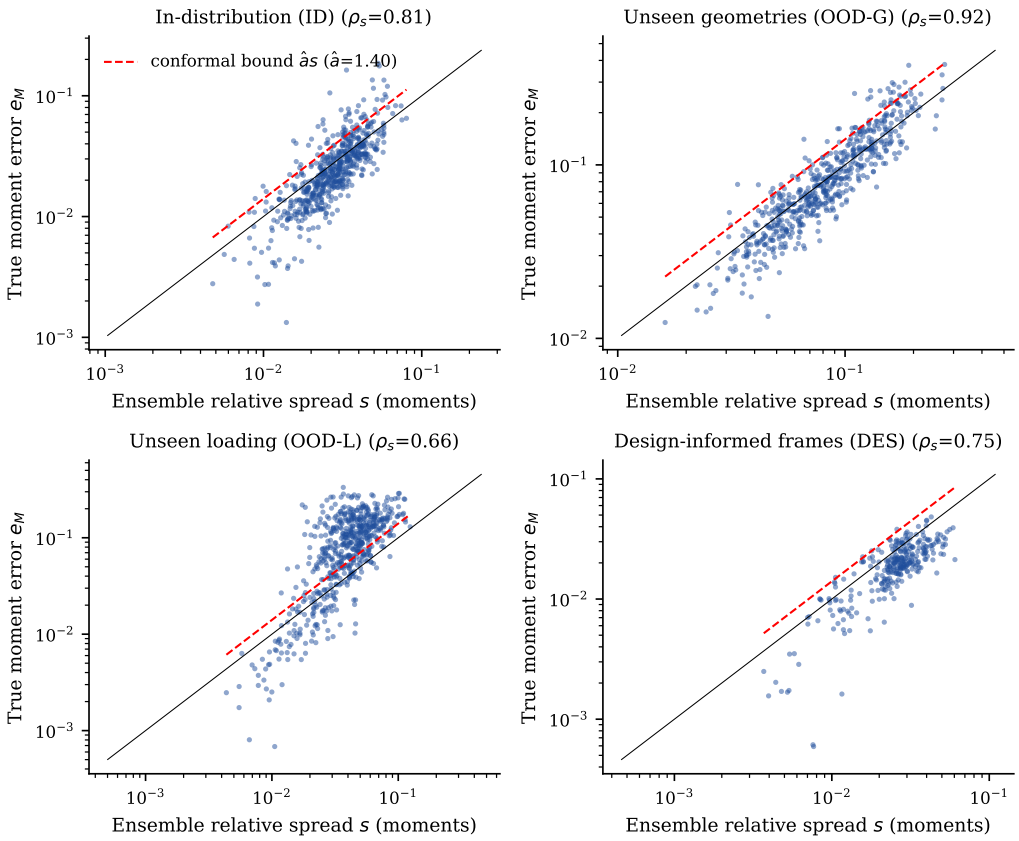


Figure 9: Frame-level ensemble spread of the end moments s (Eq. 7) against the true relative moment error for the four test sets. Black line: $e_M = s$; red dashed line: split-conformal bound $\hat{a}s$ of ensemble 0, calibrated on the conformal-calibration frames (Section 3.8).

Table 17: Two-stage referral rule with separately calibrated thresholds (mean ± standard deviation over the five independently trained ensembles; $\hat{a} = 1.40$–1.51 for $\alpha = 0.1$). Referred: by the domain check (identical for all runs), by the conformal bound $\hat{a}s > \tau$, and in total (%). Coverage: share of in-domain frames with $e_M \leq \hat{a}s$ (nominal $\geq 90\%$ for ID frames). Violation: share of the accepted frames with $e_M > \tau$ (empirical, not guaranteed). Joint: share of all frames that are accepted and have $e_M > \tau$ (bounded by $\alpha = 10\%$ for in-domain ID frames). p95: 95th percentile of e_M over the accepted frames.

τ	Set	Domain	Bound	Total	Coverage	Violation	Joint	p95 accepted
5%	ID	0.3	29.9 ± 2.9	30.2 ± 2.9	90.7 ± 0.8	2.7 ± 0.7	1.9 ± 0.6	4.3 ± 0.1
	OOD-G	48.7	48.1 ± 0.4	96.7 ± 0.4	95.1 ± 2.2	1.7 ± 3.7	0.1 ± 0.1	3.8 ± 0.7
	OOD-L	71.2	5.6 ± 0.5	76.8 ± 0.5	83.8 ± 2.0	5.7 ± 1.6	1.3 ± 0.4	5.1 ± 0.5
	DES	0.0	21.2 ± 4.6	21.2 ± 4.6	97.5 ± 1.2	0.0 ± 0.0	0.0 ± 0.0	3.1 ± 0.1
10%	ID	0.3	0.6 ± 0.1	0.9 ± 0.1	90.7 ± 0.8	1.4 ± 0.4	1.4 ± 0.4	6.3 ± 0.1
	OOD-G	48.7	24.2 ± 1.4	72.8 ± 1.4	95.1 ± 2.2	1.1 ± 0.3	0.3 ± 0.1	7.7 ± 0.5
	OOD-L	71.2	0.2 ± 0.1	71.4 ± 0.1	83.8 ± 2.0	1.7 ± 0.0	0.5 ± 0.0	6.8 ± 0.2
	DES	0.0	0.0 ± 0.0	0.0 ± 0.0	97.5 ± 1.2	0.0 ± 0.0	0.0 ± 0.0	3.6 ± 0.2

frames, and 49% of the OOD-G and 71% of the OOD-L frames. The two descriptor groups detect different shifts: the geometry descriptors flag 25% of the unseen geometries and none of the unseen load patterns, and the load-pattern descriptors flag 67% of the unseen load patterns and 10% of the unseen geometries. The load-pattern check therefore supplies exactly the information that the ensemble spread lacks. All 173 OOD-L frames that pass the check are frames whose random bay pattern happened to leave every bay fully loaded, so that only the lateral profile is new. The OOD-G frames that pass (308) include 84 six-storey frames, 83 five-bay frames with at most five storeys and 77 frames with seven or eight storeys; the domain check alone therefore does not exclude all geometric extrapolation, and Stage 2 has to refer most of these frames. No DES frame is flagged.

Compared with the rule of the first revision, in which the 400 validation frames supplied both the domain threshold and the conformal scores, the separately calibrated threshold is higher (the 99% quantile of 1,000 frames reaches further into the tail of the in-distribution distances than that of 400 frames), so fewer shifted frames are flagged (49% instead of 79% of OOD-G, 71% instead of 75% of OOD-L, none instead of 20% of DES). The violation rates below show that the conformal stage compensates for this.

Table 17 evaluates the complete rule on the untouched test sets. All numbers refer to the relative L_2 error of the end-moment field, e_M, which

is the quantity the rule is calibrated for. The conformal bound covers the true error in 90.7% of the in-domain ID test frames (five-run mean), as the guarantee requires, in 95% (OOD-G) and 98% (DES) of the in-domain frames of those sets, but only in 84% of the in-domain OOD-L frames: for shifted data the coverage is an empirical property and fails once the load pattern changes, which is why the domain check is needed. With a tolerance of $\tau = 10\%$, the rule refers 0.9% of the ID frames and accepts the rest with an observed violation rate of 1.4% (frames whose true error exceeds 10% although accepted). This conditional rate is an empirical test-set result; what the theory guarantees for ID frames is the marginal coverage above and the bound of 10% on the joint rate of accepted violations. For OOD-G, OOD-L and DES none of the rates below is guaranteed, because these frames are not exchangeable with the calibration frames. On unseen geometries it refers 73% of the frames; the accepted frames have a 95th-percentile error of 7.7% and a violation rate of 1.1%, compared with a 95th percentile of 22.6% and 35% of frames above 10% without referral. On unseen load patterns it refers 71% and the accepted frames have a violation rate of 1.7%. All 300 DES frames are accepted and none exceeds 10% (95th percentile 3.6%). With $\tau = 5\%$ the violation rates are 2.7% (ID), 1.7% (OOD-G, about 20 accepted frames), 5.7% (OOD-L) and 0% (DES), at the cost of referring 30% of the ID frames. The joint rate of accepted violations is at most 1.9% on every set and tolerance, well below the bound $\alpha = 10\%$ that holds for in-distribution frames. The rule cannot make the surrogate accurate where it is not, but it makes the referral decision before the FE result is known and with thresholds that were not tuned on the test data.

Because the rule is calibrated on e_M only, it gives no assurance for other response quantities, and Table 18 shows that this distinction matters. For the force quantities the accepted frames behave as well as, or better than, the calibrated moment field: the largest end moment exceeds a 10% error in at most 0.7% of the accepted frames, the axial forces, shears and reactions in at most 0.6%. For the kinematic quantities they do not: the largest inter-storey drift is more than 10% wrong in 19–38% of the accepted frames, the rotations in 6–23% and the horizontal displacements in 9–36%, because the moment spread says little about the error of the displacement head. The referral rule is therefore a rule for the end-moment field (and, empirically, for the other member forces) only; displacement, rotation and drift predictions of accepted frames are separately validated estimates (Table 7) without a referral guarantee, and would need their own calibrated score.

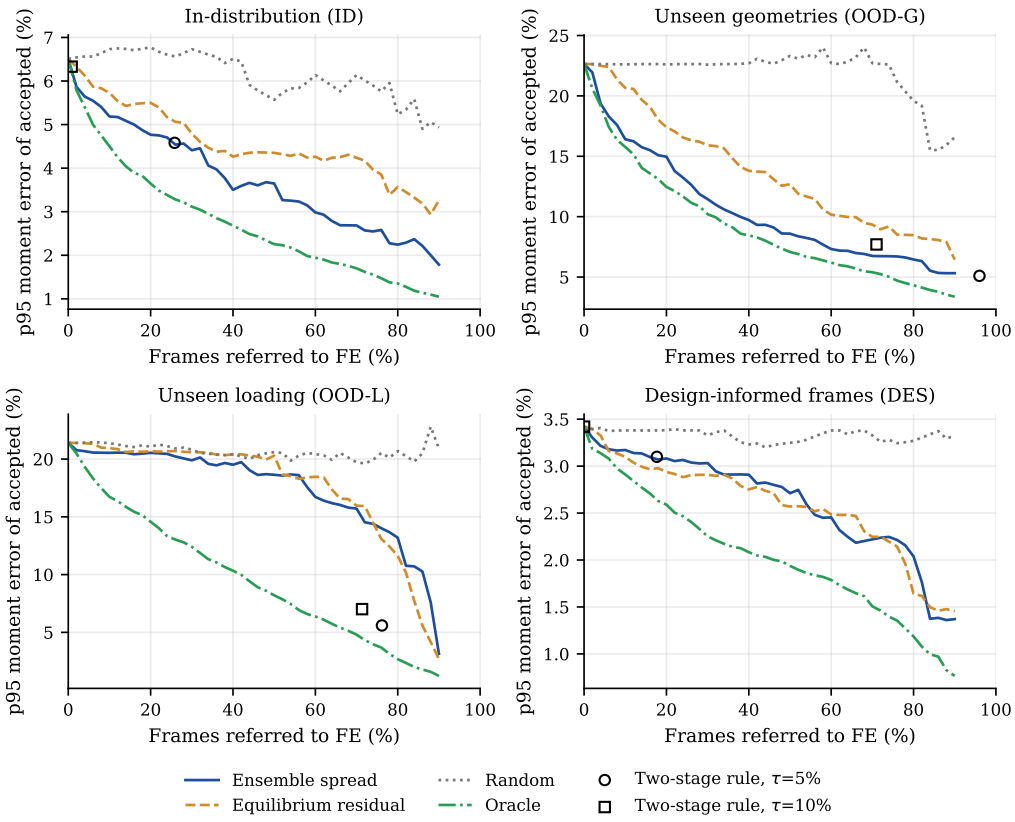


Figure 10: Risk–coverage curves: 95th percentile of the moment error over the accepted predictions when the frames with the largest scores are referred to PyNite, for the ensemble spread, the equilibrium residual, random referral and an oracle ranking by the true error (ensemble 0). Markers: operating points of the two-stage rule of ensemble 0 (Table 17). The curves are computed on the test sets and serve only to describe the ranking quality; they are not used to set any threshold.

Table 18: Scope of the referral rule: share of the frames accepted at $\tau = 10\%$ (%, mean $\pm$ standard deviation over five runs) whose error in each response quantity exceeds 10%. Only the first row is the quantity the rule is calibrated for; the other rows are empirical and carry no guarantee.

Quantity	ID	OOD-G	OOD-L	DES
e_M (calibrated)	1.4 ± 0.4	1.1 ± 0.3	1.7 ± 0.0	0.0 ± 0.0
$e_{M,\max}$	0.1 ± 0.1	0.7 ± 0.7	0.5 ± 0.3	0.0 ± 0.0
$e_{N,\max}$	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
e_N	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
e_V	0.2 ± 0.1	0.0 ± 0.0	0.6 ± 0.4	0.0 ± 0.0
e_R	0.2 ± 0.1	0.0 ± 0.0	0.1 ± 0.3	0.0 ± 0.0
e_{u_x}	11.1 ± 1.2	35.9 ± 0.7	14.2 ± 2.0	8.5 ± 0.6
e_θ	8.1 ± 0.4	22.7 ± 2.1	9.9 ± 1.1	5.5 ± 1.0
e_{drift}	21.6 ± 1.4	37.6 ± 2.1	20.1 ± 2.5	18.8 ± 1.9
e_{defl}	1.1 ± 0.5	1.4 ± 0.3	6.5 ± 1.6	0.7 ± 0.0

Figure 10 complements the rule with risk–coverage curves. On the ID set, referring the 20% of frames with the largest spread lowers the 95th-percentile moment error from 6.5% to 4.8% and catches 78% of the 1.5% of frames with errors above 10%. On unseen geometries it lowers the tail from 22.6% to 15.0%; on unseen load patterns the spread-based ranking is little better than random (21.4%→20.5%), which is why the domain check is needed.

6.9. Adaptive FE data generation

The adaptive study starts from 200 randomly chosen PyNite analyses and adds 200 per round up to 1,200, training a three-network ensemble (100 epochs) in every round. Each acquisition rule is repeated with five random initial sets (seeds 0–4; the initial set and the training seeds of a given seed are the same for all rules). Figure 11 shows the mean and standard deviation over the five repetitions and Table 19 the errors at the final budget. Because these ensembles are smaller and trained on fewer frames for fewer epochs, their errors are higher than those of the main model.

Acquiring frames by the equilibrium residual of the raw prediction gives the lowest errors on every test set, and the advantage is largest where it matters most, on unseen geometries: after 1,200 analyses the median moment error is $12.1 \pm 0.7\%$ against $14.8 \pm 0.2\%$ for random sampling, and the 95th percentile $29.5 \pm 4.2\%$ against $36.2 \pm 1.4\%$. Equilibrium-residual acquisition reaches the final median error of random sampling after 600 analyses on

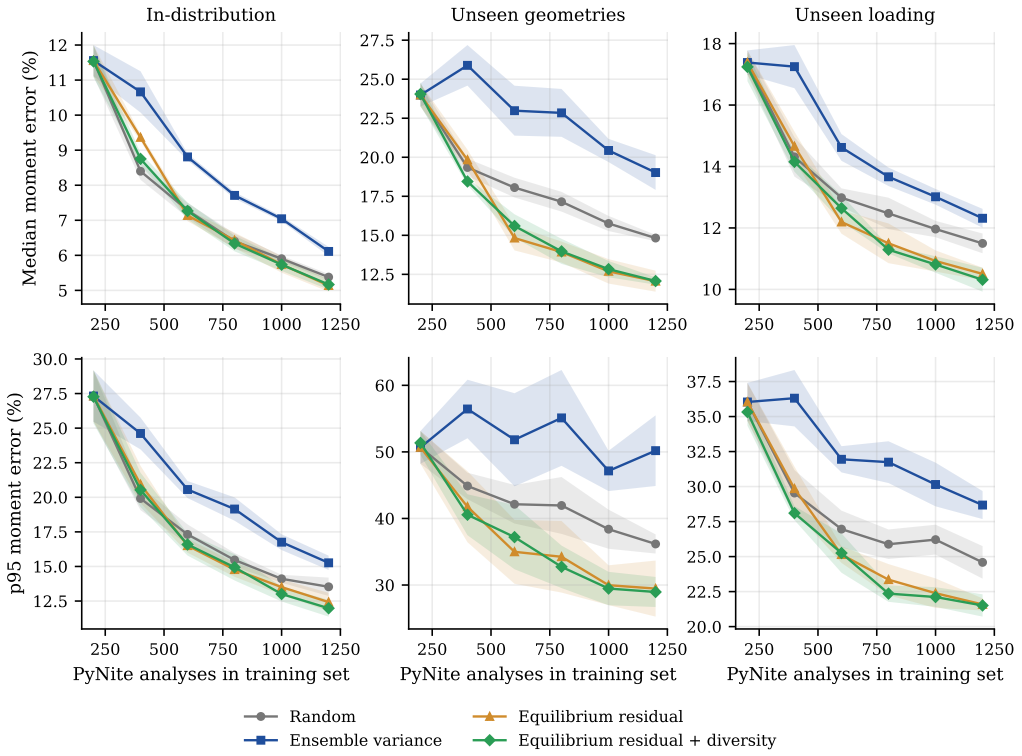


Figure 11: Moment error (median, top; 95th percentile, bottom) versus the number of PyNite analyses used for training, for four acquisition rules. Lines: mean over five repetitions; bands: ± one standard deviation. Predictions include the variance-weighted projection.

Table 19: Moment errors (%) after 1,200 PyNite analyses (mean ± standard deviation over five repetitions; number of repetitions in brackets) and number of analyses after which the mean median error first reaches the final error of random sampling (–: not reached).

	In-distribution			Unseen geometries			Unseen loading		
Acquisition rule	e_M	e_M p95	n	e_M	e_M p95	n	e_M	e_M p95	n
Random (5)	5.4 ± 0.1	13.5 ± 0.7	1200	14.8 ± 0.2	36.2 ± 1.4	1200	11.5 ± 0.3	24.6 ± 1.2	1200
Ensemble variance (5)	6.1 ± 0.1	15.3 ± 0.5	–	19.0 ± 1.1	50.2 ± 5.3	–	12.3 ± 0.3	28.7 ± 1.0	–
Equilibrium residual (5)	5.1 ± 0.2	12.4 ± 0.7	1200	12.1 ± 0.7	29.5 ± 4.2	600	10.5 ± 0.2	21.6 ± 0.4	1000
Equilibrium residual + diversity (5)	5.2 ± 0.1	12.0 ± 0.6	1200	12.1 ± 0.2	29.0 ± 2.3	800	10.3 ± 0.4	21.5 ± 0.8	800

unseen geometries and after 1,000 on unseen loading, i.e. with half and five sixths of the FE budget (the diversity-aware variant after 800 in both cases); on the ID set the gain is small ($5.1 \pm 0.2\%$ against $5.4 \pm 0.1\%$) and the random-sampling level is reached only at the final budget. Acquisition by the ensemble variance is worse than random sampling on all sets (OOD-G $19.0 \pm 1.1\%$, p95 $50 \pm 5\%$). The acquired frames explain the difference. Compared with random sampling (on average 3.0 storeys, 2.5 bays, 70% fixed bases; seeds 0 and 2), variance acquisition picks small frames (2.6–2.7 storeys, 1.9 bays) with many pinned bases (50%), for which the dimensionless outputs vary most between ensemble members but which teach little about larger frames. Residual acquisition picks taller and more flexible frames (4.1–4.2 storeys, 55% fixed bases, median largest drift ratio 1.6% against 0.5%), i.e. the frames whose force predictions are furthest from equilibrium, and these are also the most informative for the unseen taller geometries. The diversity-aware variant gives the same median errors as plain residual acquisition (OOD-G $12.1 \pm 0.2\%$, OOD-L $10.3 \pm 0.4\%$) with a smaller spread between repetitions and slightly lower 95th percentiles; at a batch size of 200 from a pool of 5,000, redundancy within a batch is therefore not the limiting factor. These differences are larger than the variation between repetitions, but with five repetitions and one pool they should be read as evidence for this frame family, not as a general ranking of acquisition rules.

6.10. Computational cost

Timings were measured serially on the otherwise idle machine (Section 5), five times for 200 frames of each test set (Table 20). Besides PyNite, two other solvers were timed on the same frames for both load cases: OpenSeesPy and the vectorised sparse direct-stiffness solver written for the verification (Section 2.3), which represents an optimised minimal FE pipeline for this problem class. All three FE timings have the same scope: each starts from the

Table 20: Wall-clock time per frame (ms; mean ± standard deviation over five repetitions of 200 frames, 4-core 2.1 GHz CPU, networks evaluated with 4 threads): PyNite, OpenSeesPy and the sparse direct-stiffness solver (DSM) for both load cases; the proposed surrogate evaluated in batches and for one frame alone; batched speed-up relative to PyNite and to the DSM solver.

Set	PyNite	OpenSeesPy	DSM	Surrogate (batched)	Surrogate (single)	vs PyNite	vs DSM
ID	33.55 ± 1.58	1.19 ± 0.26	1.50 ± 0.22	2.71 ± 0.14	20.24 ± 7.48	12.4	0.56
OOD-G	98.04 ± 3.96	2.78 ± 0.13	2.38 ± 0.13	6.99 ± 0.31	87.39 ± 3.93	14.0	0.34
OOD-L	34.60 ± 0.65	1.37 ± 0.19	1.78 ± 0.16	2.75 ± 0.25	30.32 ± 4.01	12.6	0.65

frame parameters and includes model construction, assembly, solution and the extraction of nodal displacements and member end forces for both load cases. PyNite builds one model with two load cases; OpenSeesPy and the DSM solver build and solve the model once per load case (with `wipe` between cases for OpenSeesPy), so if anything their timings are slightly pessimistic. The surrogate timing likewise starts from the frame parameters and includes feature and graph construction, the ensemble evaluation and the projection. A PyNite analysis of one frame (two load cases, including model construction and result extraction) costs 33.6 ± 1.6 ms for ID frames and 98.0 ± 4.0 ms for the larger OOD-G frames. Evaluated in batches, the proposed surrogate costs 2.7 ms (ID and OOD-L) and 7.0 ms (OOD-G) per frame, including the Python feature construction (the largest part), the five-network ensemble and the projection (0.1–0.6 ms), i.e. 12–14 times less than PyNite. For a single frame evaluated in isolation the overhead of the network framework dominates and the surrogate (20–87 ms) is not faster than PyNite.

The comparison with an optimised solver changes the picture. OpenSeesPy and the sparse direct-stiffness solver analyse a frame in 1.2–2.8 ms, faster than the batched surrogate on every test set (speed-up of the surrogate over the DSM solver 0.34–0.65). Most of the cost of a PyNite analysis is the construction of the general three-dimensional model and the extraction of results through its object interface, not the solution of the linear system, which for frames with at most a few hundred DOFs takes well under a millisecond. Against PyNite, the batched surrogate pays back its one-off cost (about 41 min of CPU time to train the five networks, while the 3,000 training analyses cost about 2 min) only after roughly 8×10^4 ID frames without referral and 1.1×10^5 with 20% of the frames referred to PyNite; against an optimised solver it never does. For first-order linear analysis of frames of this size the surrogate therefore offers no speed

advantage over a well-implemented FE solver. Its value in this setting lies elsewhere: a batched, equilibrium-projected surrogate of the member forces with a calibrated referral rule for response screening, and a template for analyses that are genuinely expensive (nonlinear, dynamic, or much larger models), which this paper does not test. End-to-end differentiability with respect to the frame parameters is not claimed: the features and the equilibrium matrix $\mathbf{C}$ are assembled, and the projection is solved, in NumPy outside the automatic-differentiation graph, and the domain check, the conformal acceptance and the FE referral are discrete decisions. Use in gradient-based design would require a differentiable implementation of these steps and a verification of the gradients, which this study does not provide.

7. Discussion

Answer to the research question. The answer depends on which output is used, because the surrogate does not return a mutually compatible set of displacements and forces as an FE analysis does. Three uses should be distinguished.

1. *Member-force screening.* The directly predicted, equilibrium-projected member forces are the most reliable output. Within the training design space and on the design-informed frames the median error of the end-moment field is 2.4% (ID) and 2.0% (DES), the largest end moment, largest axial force and reactions are within 1% at the median, and the forces are exactly in equilibrium with the loads. For this use the two-stage referral rule applies, with a guarantee for in-distribution frames on the end-moment error: $P(e_M \leq \hat{a}s) \geq 0.9$, and hence $P(\text{accepted and } e_M > \tau) \leq 0.1$. In the test sets the observed coverage was 90.7% and, at $\tau = 10\%$, the rule accepted 99% of the ID frames, of which 1.4% had an end-moment error above 10%; this is an observed conditional violation rate, not a guaranteed one. On the shifted sets it referred most frames and the observed rate among the accepted frames was 1.1–1.7%, with no guarantee. Among the accepted frames, the other force quantities exceed a 10% error in at most 0.7% of the frames, but this is an empirical observation, not a guarantee.
2. *Kinematic screening.* The predicted displacements, rotations and drifts are validated separately (Table 7) and are less accurate (median 4.5%

for the horizontal displacement field, 95th percentile of the largest-drift error 20% on ID). The referral rule does not protect them: up to 38% of the accepted frames have a drift error above 10%. They support exploratory ranking, trend studies and preliminary estimation as supplementary screening estimates, but the current framework provides no calibrated decision mechanism for accepting displacement or drift predictions in place of FE analysis, and they should not be used to check a drift limit near its threshold.

3. *Complete structural analysis.* This use is not established. The projected forces are not compatible with the predicted displacements (Section 6.6), so the two outputs cannot be combined as if they came from one FE solution, for example to check a member against both its forces and its end rotations.

The current referral framework therefore directly supports member-force screening inside a validated domain, where it can reduce the number of repeated FE analyses; the kinematic outputs are supplementary screening estimates without a calibrated acceptance rule. FE analysis is retained for any final structural verification, for the frames that the rule refers and for any decision that depends on displacements or drifts. Outside the training space the median errors roughly triple and the 95th percentile of the moment error exceeds 20%; the surrogate is then usable only together with the referral rule.

What the constraints buy. The exact constraints contribute in different ways. Dimensional scaling and load-case separation are an *accuracy* device: they remove the need to learn the dependence on the load intensities and on E and make the lateral and gravity responses equally well conditioned. Used together, they halve the displacement errors of a conventional GNN with the same architecture (Table 11); the statics-derived features add a smaller benefit, mainly for the displacements under unseen load patterns. The improvement should therefore be credited to the combination of dimensionless scaling, load-case separation and statics features, and mostly to the first two, rather than to the statics features alone. The equilibrium projection is primarily an *admissibility* device. It guarantees static equilibrium, which a soft penalty does not achieve; with a variance-weighted metric it also reduces the axial-force and reaction errors by factors of 2–5, but its effect on the moment errors is small (0.1 to 0.5 percentage points on average over five runs) and statistically clear only on some test sets. With a poorly chosen metric the

projection degrades the moments, so the metric is an essential part of the method. The projection does not enforce compatibility: it slightly increases the mismatch between the predicted forces and the forces implied by the predicted displacements, and that mismatch, dominated by the axial terms, is large. Recovering forces from predicted displacements gives compatibility but loses accuracy and equilibrium, by a wide margin. Imposing both conditions exactly amounts to solving the FE problem; using the predicted displacements as a warm start of an iterative solver did not help either.

Architecture. A StructGNN-style network with storey pseudo nodes, given the same inputs and targets, reached similar accuracy, better for displacements on unseen geometries and slightly worse for moments, at seven times the training cost; global pooling and deeper message passing gave the proposed network the same displacement gains. The architecture of the learned model is therefore not the main contribution of this work. The exact constraints, the uncertainty estimate and the decision rules built on them are model-agnostic and could be wrapped around any of these networks.

Uncertainty as a decision tool. The two uses of uncertainty have distinct indicators. At deployment, the ensemble spread is a good ranking of the errors for in-distribution, geometrically extrapolated and design-informed frames, but under a change of load pattern the ensemble is confidently wrong (coverage of the $\pm 2\sigma$ intervals 67% instead of 92%). A domain check on simple load-pattern descriptors, which are known before any analysis, detects most of those frames, and a split-conformal bound calibrated on validation data turns the spread into a threshold with a stated coverage. During training-set construction, acquisition by ensemble variance was worse than random sampling, whereas acquisition by the equilibrium residual of the raw prediction, which requires no labels, was the best rule in all five repetitions and reached the accuracy of random sampling on unseen geometries with half the FE analyses. Diversity-aware batch selection did not improve the median errors further but reduced the variation between repetitions.

Cost. The batched surrogate is 12–14 times faster than PyNite per frame, but an optimised sparse stiffness solver and OpenSeesPy analyse the same frames 1.5–3 times faster than the surrogate. For cheap first-order linear analyses the speed argument therefore does not hold; it was the main justification in the original formulation of this study and has been revised. The accuracy,

admissibility and referral results do not depend on it, and they are the parts of the framework that carry over to expensive analyses.

Limitations. The study is restricted to planar, regular, multi-storey moment frames with prismatic members, rigid joints, uniformly distributed beam loads and floor-level lateral forces, analysed with first-order linear elasticity and a uniform Young's modulus. The design-informed set uses simplified sizing rules, not code-compliant designs, and it shares the topology family of the training data. The graph baselines are re-implementations from the published descriptions with adaptations stated in Section 5; in particular, the poor performance of the stiffness-residual loss in our Leng-style implementation may reflect our scaling of the residual rather than the original method. The adaptive study is retrospective: candidates were labelled in advance and their labels revealed only on request, which gives exact budget accounting but not the scheduling overheads of a live FE loop. The conformal guarantee assumes exchangeability with the calibration frames and holds only approximately for the in-domain part of shifted distributions. Timings were measured on one CPU with Python implementations of the surrogate and of all three solvers.

Outlook. Natural extensions are a mixed formulation that also enforces compatibility through the member flexibilities, three-dimensional frames, braced systems and second-order (P-Δ) analysis, for which the FE cost is higher and superposition no longer holds exactly, and conformal methods that adapt to covariate shift.

8. Conclusions

Equilibrium-projected graph neural network surrogates for the first-order linear analysis of planar moment frames were developed and evaluated on 9,500 PyNite analyses, verified against an independent stiffness solver, OpenSeesPy and closed-form solutions. A graph network ensemble predicts nodal displacements and member basic forces in a dimensionless, load-case-separated form, so that superposition, homogeneity in the loads, the $1/E$ scaling of displacements and the boundary conditions hold exactly; an equilibrium projection with a variance-weighted metric makes the predicted forces statically admissible; a domain check and a conformal bound on the ensemble spread decide which frames are referred to PyNite at deployment;

and the equilibrium residual of the raw prediction selects new FE analyses during training. The main conclusions are:

1. On in-distribution and design-informed frames the surrogate reaches median end-moment errors of 2.4% and 2.0% (five independent runs: $2.37 \pm 0.03\%$ and $1.99 \pm 0.08\%$), axial-force errors of 0.6% and 0.3%, and horizontal displacement errors of 4.5% and 4.4%, four to nine times lower than the best of five tabular baselines given identical features.
2. On unseen geometries and unseen load patterns the median moment errors are 7.5% and 8.7% (best tabular baseline 22.1% and 16.8%), but the 95th percentiles exceed 20%.
3. The combined use of dimensionless scaling and load-case separation halves the displacement errors of a conventional graph network; statics-derived features add a smaller, mainly displacement-related benefit. A StructGNN-style network performs similarly to the proposed network, so the benefits of the framework lie in its exact constraints and decision rules rather than in the network architecture. Displacement-only networks with force recovery give much less accurate forces.
4. The equilibrium projection reduces the out-of-balance force from 4–16% (raw ensemble) and up to 120% (baselines) to below 10^{-12} for every model, reduces the axial-force errors by factors of 2–5 and lowers the median moment errors by 0.1–0.5 percentage points on average over five runs with a variance-weighted metric, whereas a Euclidean metric increases them. It slightly increases the displacement–force incompatibility.
5. A two-stage referral rule, with the domain threshold and the conformal quantile calibrated on two separate sets of new frames, covers the end-moment error of in-distribution test frames in 90.7% of cases, consistent with the guaranteed marginal coverage of at least 90%. At a 10% tolerance it accepts 99% of the ID frames; the observed (empirical, not guaranteed) share of accepted frames with an end-moment error above 10% is 1.4%. It refers 71–73% of the unseen frames, and the observed share among the accepted ones is 1.1–1.7%, again without a guarantee because these frames are not exchangeable with the calibration data. The rule is calibrated for the end-moment field only: the other member forces of accepted frames were also accurate, but the drift, rotation and displacement errors of up to 38% of the accepted frames exceeded 10%.
6. Acquiring training frames by the equilibrium residual of the raw pre-

diction reached the accuracy of 1,200 random analyses on unseen geometries with 600 analyses in five repetitions, whereas acquisition by ensemble variance was worse than random sampling.

7. The batched surrogate is 12–14 times faster than a PyNite analysis per frame but 1.5–3 times slower than an optimised sparse stiffness solver, so for first-order linear analyses of frames of this size it offers no speed advantage over a well-implemented FE solver; its value lies in admissible, uncertainty-aware batched predictions and as a template for more expensive analyses.

The framework is best viewed as an equilibrium-enforced, uncertainty-aware surrogate for selected structural-response screening tasks, in particular member-force screening, that knows the limits of its training data. It is not a replacement for a complete FE analysis, whose displacements and forces are mutually compatible, and FE analysis remains necessary for final structural verification.

Data and code availability

All scripts used to generate and verify the PyNite dataset, train the models, run the experiments and produce every figure and table are in the `research/surrogate` folder of the development branch `claude/project-thread-014zmy` of the PyNite fork (`https://github.com/karthice1/Pynite`); at the time of writing this folder has not yet been merged into the default branch and no versioned release has been published. The package includes the independent direct-stiffness solver (`solvers.py`), the OpenSeesPy verification (`experiment_validation.py`), the input parameters of all 9,500 frames (`release/frame_parameters.json.gz`) with a script that re-runs the cross-solver verification directly from them (`verify_from_parameters.py`), pinned package versions (`requirements.txt`), the weights of the reported ensemble (`weights/`) and machine-readable results of every table. `reproduce.sh` reruns every experiment from scratch and is the definitive reproduction route; its `--quick` option only regenerates the report from the bundled weights and committed result summaries and is not an independent reproduction of the multi-run experiments. The datasets are regenerated deterministically from the seeds in the scripts. [Before publication the authors will merge the folder into the default branch, publish a versioned release and archive it with the

dataset on Zenodo; the release tag, commit hash and DOI will be inserted here.]

CRediT authorship contribution statement

To be completed by the authors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used an AI assistant (Claude, Anthropic) to write the analysis code, run the experiments and draft the manuscript. All numerical results were produced by the scripts of the reproducibility package. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Appendix A. Feature dictionary

Table A.21 lists the node and member features of the proposed network. Notation: node k at floor level l (0 at the base) on column line c; $sb = l - 1$ and $sa = l$ are the storeys below and above the node; $s_{\mathrm{ref}} = sa$ if $l < n_s$ and sb otherwise; λ_{W}, λ_{G}, H, I_{ref} as in Section 3.2; $u_0 = \lambda_{\mathrm{W}} H^3/(E I_{\mathrm{ref}})$. A feature that refers to a missing neighbour (e.g. the column below a base node) is set to zero. For the combined-case ablation, λ_{W} and λ_{G} are both replaced by $|\lambda_{\mathrm{W}}| + \lambda_{\mathrm{G}}$.

Appendix B. Sampling procedure

Each frame of the ID, OOD-G and OOD-L sets is drawn with its own seed by the following steps (`frames.sample_frame`); the DES frames are generated as described in Appendix C.

Table A.21: Feature dictionary of the proposed network (25 node and 10 member features). Exact: statically exact quantity. Approx.: classical approximation.

Name	Definition	Type
Node features		
`lev`, `line`	l/n_s, $c/\max(n_b,1)$	topology
`is_base`, `is_ext`	$[l=0]$, $[c\in\{0,n_b\}]$	topology
`x`, `y`	X_k/H, Y_k/H	geometry
`n_st`, `n_bay`	$n_s/5$, $n_b/4$	topology
`fixed`	1 for fixed bases, 0 for pinned	supports
`Px`	$P_{x,k}/\lambda_{\mathrm{W}}$	load
`V_above`	$V_{s_{\mathrm{ref}}}/\lambda_{\mathrm{W}}$, $V_s=\sum_{k\geq s}F_k$	exact statics
`OTM_above`	$\sum_{k\geq s}F_k(Y_{k+1}-Y_s)/(\lambda_{\mathrm{W}}H)$ at $s=s_{\mathrm{ref}}$	exact statics
`G_above`	tributary gravity load of column line c from floors $\geq s_{\mathrm{ref}}+1$, $/\lambda_{\mathrm{G}}$	approx. (simply supported beams)
`Ic_below`, `Ic_above`	$I_{c,sb}/I_{\mathrm{ref}}$, $I_{c,sa}/I_{\mathrm{ref}}$	section
`Ib_left`, `Ib_right`	$I_{b,sb}/I_{\mathrm{ref}}$ if a beam frames in from the left / right	section
`Lb_left`, `Lb_right`	bay widths left / right of the line, $/H$	geometry
`h_below`, `h_above`	h_{sb}/H, h_{sa}/H	geometry
`rho_level`	$\rho_{s_{\mathrm{ref}}}=(n_bI_{b,s}/\bar{L}_b)/((n_b+1)I_{c,s}/h_s)$	approx. (stiffness ratio)
`drift_sb`	$\delta_{s_{\mathrm{ref}}}/u_0$, $\delta_s=V_s/k_s$, $k_s=(n_b+1)12EI_{c,s}/h_s^3$ ($3EI/h^3$ pinned base)	approx. (shear building)
`disp_sb`	$\sum_{k\leq sb}\delta_k/u_0$	approx. (shear building)
`w_trib`	tributary gravity load of floor l on line c, $/\lambda_{\mathrm{G}}$	approx.
Member features		
`is_col`, `is_beam`	member type indicators	topology
`storey`	s/n_s	topology
`L`, `I`, `A`	L/H, I/I_{ref}, AH^2/I_{ref}	geometry, section
`wL`	$w_eL_e/\lambda_{\mathrm{G}}$ (signed)	load
`rho_storey`, `V_storey`, `drift_sb`	ρ_s, V_s/λ_{W}, δ_s/u_0 of the member's storey	as above

1. Topology. ID and OOD-L: $n_s\sim\mathcal{U}\{1,\dots,5\}$, $n_b\sim\mathcal{U}\{1,\dots,4\}$. OOD-G: with probability 0.5, $n_s\sim\mathcal{U}\{6,7,8\}$ and $n_b\sim\mathcal{U}\{1,\dots,6\}$; otherwise $n_s\sim\mathcal{U}\{1,\dots,8\}$ and $n_b\sim\mathcal{U}\{5,6\}$.
2. Geometry. $L_b\sim\mathcal{U}(3,9)$ per bay and $h_s\sim\mathcal{U}(2.8,4.5)$ per storey; OOD-G replaces h_1 by $\mathcal{U}(4.5,6.0)$ with probability 0.5.
3. Material and sections. $E\sim\log\mathcal{U}(2.5\times10^7,2.1\times10^8)$; base column $I_{c,1}\sim\log\mathcal{U}(5\times10^{-5},2\times10^{-3})$ with $I_{c,s}=I_{c,1}t^{(s-1)/\max(n_s-1,1)}$, $t\sim\mathcal{U}(0.5,1)$; $I_{b,s}\sim\log\mathcal{U}(5\times10^{-5},2\times10^{-3})$ independently per storey; $r_c,r_b\sim\mathcal{U}(0.08,0.30)$ and $A=I/r^2$.
4. Supports. Fixed bases with probability 0.7, pinned otherwise.
5. Gravity load. $w_s\sim\mathcal{U}(5,40)$ per floor on all bays. OOD-L: each bay keeps w_s with probability 0.5 and otherwise only a dead part d_sw_s, $d_s\sim\mathcal{U}(0.3,0.6)$.
6. Lateral load. Roof force $F_{\mathrm{roof}}\sim\mathcal{U}(5,80)$; profile uniform or inverted triangular (ID, OOD-G) or parabolic or roof-only (OOD-L), each with equal probability; the sign of all floor forces is reversed with probability 0.5.
7. Analysis. PyNite linear analysis of load cases W and G; the statics check of Section 2 must pass.

Appendix C. Design-informed synthetic frames (DES)

Each of the 300 DES frames is generated with its own seed ($9 \times 10^6 + k$, $k = 0, \ldots, 299$; `frames.sample_design_frame`, called by `experiment_validation.py`) as follows. Units are kN and m; $\mathcal{U}$ denotes a uniform distribution.

1. Topology. $n_s \sim \mathcal{U}\{1, \ldots, 5\}$, $n_b \sim \mathcal{U}\{1, \ldots, 4\}$ (realised counts for 1–5 storeys: 58, 62, 64, 50, 66; for 1–4 bays: 74, 76, 73, 77).
2. Geometry. Each bay width is drawn from $\{6.0, 7.5, 9.0\}$ with equal probability; all storeys above the first have the same height $h \sim \mathcal{U}(3.2, 3.8)$ and the first storey $h_1 \sim \mathcal{U}(4.0, 4.5)$.
3. Gravity load. Tributary width $b \sim \mathcal{U}(4, 6)$, dead load $D \sim \mathcal{U}(3.5, 5.0)\,\mathrm{kPa}$, live load $L_q \sim \mathcal{U}(2.0, 3.0)\,\mathrm{kPa}$. Floor line load $w = (1.2D + 1.0L_q)\,b$ on every bay; roof line load $w_{\mathrm{roof}} = (1.2(D - 1.0) + 1.0 \cdot 1.0)\,b$.
4. Lateral load. Wind pressure $q_w \sim \mathcal{U}(0.6, 1.2)\,\mathrm{kPa}$; the force at floor s is $F_s = q_w\,b\,(h_s + h_{s+1})/2$ with $h_{n_s+1} = 0$; all forces change sign with probability 0.5.
5. Beam sizing (one section for all floors). With $L_{\max}$ the largest bay, $I_{\mathrm{defl}} = 5L_q b L_{\max}^4/(384E\,L_{\max}/360)$ (live-load deflection $L/360$) and $I_{\mathrm{str}} = 0.15\,(w_{\max}L_{\max}^2/8)/f_y$ with $f_y = 250\,\mathrm{MPa}$ (simple-span moment, section depth about $0.3\,\mathrm{m}$); $I_b = \max(I_{\mathrm{defl}}, I_{\mathrm{str}})$.
6. Column sizing. Storey shear $V_s = \sum_{k \geq s} |F_k|$; $I_{c,s} = V_s h_s^3/(0.5 \cdot 12E(n_b + 1)\,h_s/400)$, i.e. the shear-building drift equals $h_s/400$ with the column stiffness halved for beam flexibility; within each pair of storeys $(1, 2), (3, 4), \ldots$ the larger value is used for both; finally $I_{c,s} \geq 0.3 I_b$.
7. Clipping. I_c and I_b are clipped to $[5 \times 10^{-5}, 2 \times 10^{-3}]\,\mathrm{m}^4$, the training range (28% of the frames have at least one clipped column section, 0.3% a clipped beam section). Areas follow from radii of gyration $r_c = 0.16\,\mathrm{m}$ and $r_b = 0.18\,\mathrm{m}$, $A = I/r^2$.
8. Supports. Fixed bases with probability 0.8, pinned otherwise.
9. Analysis. PyNite linear analysis of cases W and G; the statics check of Section 2 must pass. No frame is rejected.

Table D.22: Median relative errors (%) of the tabular baselines with 95% frame-bootstrap confidence intervals (2,000 resamples) on the four test sets.

	ID		OOD-G		OOD-L		DES	
Model	e_{u_x}	e_M	e_{u_x}	e_M	e_{u_x}	e_M	e_{u_x}	e_M
Linear ridge	53.5 [49.7, 58.6]	38.1 [36.6, 39.2]	61.4 [54.1, 66.2]	82.2 [77.3, 87.0]	52.8 [48.6, 56.9]	44.3 [42.7, 46.1]	118.3 [112.6, 122.5]	35.5 [33.7, 37.2]
Quadratic ridge	51.0 [46.1, 55.1]	18.4 [17.4, 19.0]	66.6 [60.8, 73.3]	44.6 [42.1, 48.0]	53.2 [49.3, 56.9]	26.7 [25.8, 27.4]	93.3 [89.5, 98.9]	14.7 [14.1, 15.5]
Random forest	33.6 [31.3, 36.0]	16.9 [16.1, 17.5]	41.9 [39.5, 43.9]	32.5 [31.1, 33.5]	33.8 [31.8, 35.7]	24.2 [23.5, 25.1]	42.9 [39.9, 45.5]	13.5 [12.9, 14.1]
Gradient boosting	29.0 [26.4, 31.3]	10.0 [9.6, 10.4]	43.1 [41.3, 45.5]	22.5 [21.3, 23.5]	28.4 [25.9, 31.5]	17.3 [16.7, 17.8]	37.3 [33.0, 40.7]	8.7 [8.3, 9.1]
MLP	39.5 [37.5, 42.1]	10.8 [10.1, 11.2]	48.5 [43.7, 53.1]	22.1 [20.9, 22.7]	38.0 [36.5, 40.0]	16.8 [16.1, 17.7]	45.2 [41.4, 52.7]	8.7 [8.4, 9.1]

Appendix D. Confidence intervals of the tabular baselines

References

[1] A. I. J. Forrester, A. Sóbester, A. J. Keane, Engineering Design via Surrogate Modelling: A Practical Guide, John Wiley & Sons, Chichester, 2008. doi:10.1002/9780470770801.

[2] B. J. Bichon, M. S. Eldred, L. P. Swiler, S. Mahadevan, J. M. McFarland, Efficient global reliability analysis for nonlinear implicit performance functions, AIAA Journal 46 (10) (2008) 2459–2468. doi:10.2514/1.34321.

[3] B. Echard, N. Gayton, M. Lemaire, AK-MCS: An active learning reliability method combining Kriging and Monte Carlo simulation, Structural Safety 33 (2) (2011) 145–154. doi:10.1016/j.strusafe.2011.01.002.

[4] Y. Xie, M. Ebad Sichani, J. E. Padgett, R. DesRoches, The promise of implementing machine learning in earthquake engineering: A state-of-the-art review, Earthquake Spectra 36 (4) (2020) 1769–1801. doi:10.1177/8755293020919419.

[5] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, Nature Reviews Physics 3 (6) (2021) 422–440. doi:10.1038/s42254-021-00314-5.

[6] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, Journal of Computational Physics 378 (2019) 686–707. doi:10.1016/j.jcp.2018.10.045.

[7] E. Haghighat, M. Raissi, A. Moure, H. Gomez, R. Juanes, A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics, Computer Methods in Applied Mechanics and Engineering 379 (2021) 113741. doi:10.1016/j.cma.2021.113741.

[8] R. Zhang, Y. Liu, H. Sun, Physics-guided convolutional neural network (PhyCNN) for data-driven seismic response modeling, Engineering Structures 215 (2020) 110704. doi:10.1016/j.engstruct.2020.110704.

[9] R. Zhang, Y. Liu, H. Sun, Physics-informed multi-LSTM networks for metamodeling of nonlinear structures, Computer Methods in Applied Mechanics and Engineering 369 (2020) 113226. doi:10.1016/j.cma.2020.113226.

[10] T. Beucler, M. Pritchard, S. Rasp, J. Ott, P. Baldi, P. Gentine, Enforcing analytic constraints in neural networks emulating physical systems, Physical Review Letters 126 (9) (2021) 098302. doi:10.1103/PhysRevLett.126.098302.

[11] L. Lu, R. Pestourie, W. Yao, Z. Wang, F. Verdugo, S. G. Johnson, Physics-informed neural networks with hard constraints for inverse design, SIAM Journal on Scientific Computing 43 (6) (2021) B1105–B1132. doi:10.1137/21M1397908.

[12] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, et al., Relational inductive biases, deep learning, and graph networks, arXiv preprint arXiv:1806.01261 (2018).

[13] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, G. E. Dahl, Neural message passing for quantum chemistry, in: Proceedings of the 34th International Conference on Machine Learning, Vol. 70 of Proceedings of Machine Learning Research, 2017, pp. 1263–1272.

[14] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, P. W. Battaglia, Learning to simulate complex physics with graph networks, in: Proceedings of the 37th International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, 2020, pp. 8459–8468.

[15] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, P. W. Battaglia, Learning mesh-based simulation with graph networks, in: International Conference on Learning Representations (ICLR), 2021.

[16] M. Maurizi, C. Gao, F. Berto, Predicting stress, strain and deformation fields in materials and structures with graph neural networks, Scientific Reports 12 (2022) 21834. doi:10.1038/s41598-022-26424-3.

[17] E. Whalen, C. Mueller, Toward reusable surrogate models: Graph-based transfer learning on trusses, Journal of Mechanical Design 144 (2) (2022) 021704. doi:10.1115/1.4052298.

[18] B. Lakshminarayanan, A. Pritzel, C. Blundell, Simple and scalable predictive uncertainty estimation using deep ensembles, in: Advances in Neural Information Processing Systems, Vol. 30, 2017.

[19] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. V. Dillon, B. Lakshminarayanan, J. Snoek, Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift, in: Advances in Neural Information Processing Systems, Vol. 32, 2019.

[20] B. Settles, Active learning literature survey, Computer Sciences Technical Report 1648, University of Wisconsin–Madison, Department of Computer Sciences (2009).

[21] F. Parisi, S. Ruggieri, R. Lovreglio, M. P. Fanti, G. Uva, On the use of mechanics-informed models to structural engineering systems: Application of graph neural networks for structural analysis, Structures 59 (2024) 105712. doi:10.1016/j.istruc.2023.105712.

[22] Y.-T. Chou, W.-T. Chang, J. G. Jean, K.-H. Chang, Y.-N. Huang, C.-S. Chen, StructGNN: An efficient graph neural network framework for static structural analysis, Computers & Structures 299 (2024) 107385. doi:10.1016/j.compstruc.2024.107385.

[23] Z. Liang, J. Li, L. Deng, X. Kong, PeTIGN: A physics-encoded graph network for dynamic response computation of large-scale structures, Engineering Structures 364 (2026) 123094. doi:10.1016/j.engstruct.2026.123094.

[24] L. Patrignani, S. T. Pinho, Graph neural networks with hybrid local-global attention for effective prediction of mechanical response in structures, Computer Methods in Applied Mechanics and Engineering 452 (2026) 118753. doi:10.1016/j.cma.2026.118753.

[25] S.-P. Kuo, P. D'Acunto, I. F. C. Smith, Graph neural networks for fast structural analysis to support interactive design decision-making, in: A. Moreno-Rangel, B. Kumar (Eds.), Proceedings of the International Workshop on Intelligent Computing in Engineering (EG-ICE 2025), University of Strathclyde Publishing, Glasgow, 2025, pp. 664–672. doi:10.17868/strath.00093287.

[26] K. Leng, P. He, H. Jiang, Q. Feng, W. Zheng, Y. Zhou, Mechanically constrained graph neural networks for linear static analysis of planar frame structures with member loads, Buildings 16 (13) (2026) 2530. doi:10.3390/buildings16132530.

[27] D. C. Brinck, PyNite: A simple 3D structural finite element library for Python, https://github.com/JWock82/PyNite, version 3.2.0 (PyNite-FEA), fork used in this study: https://github.com/karthice1/Pynite (2026).

[28] M. Zhu, F. McKenna, M. H. Scott, OpenSeesPy: Python library for the OpenSees finite element framework, SoftwareX 7 (2018) 6–11. doi:10.1016/j.softx.2017.10.009.

[29] W. McGuire, R. H. Gallagher, R. D. Ziemian, Matrix Structural Analysis, 2nd Edition, John Wiley & Sons, New York, 2000.

[30] E. Buckingham, On physically similar systems; illustrations of the use of dimensional equations, Physical Review 4 (4) (1914) 345–376. doi:10.1103/PhysRev.4.345.

[31] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, in: International Conference on Learning Representations (ICLR), 2019.

[32] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, et al., PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, Vol. 32, 2019.

[33] J. Nocedal, S. J. Wright, Numerical Optimization, 2nd Edition, Springer, New York, 2006. doi:10.1007/978-0-387-40065-5.

[34] A. Der Kiureghian, O. Ditlevsen, Aleatory or epistemic? Does it matter?, Structural Safety 31 (2) (2009) 105–112. doi:10.1016/j.strusafe.2008.06.020.

[35] K. Lee, K. Lee, H. Lee, J. Shin, A simple unified framework for detecting out-of-distribution samples and adversarial attacks, in: Advances in Neural Information Processing Systems 31 (NeurIPS 2018), 2018.

[36] O. Ledoit, M. Wolf, A well-conditioned estimator for large-dimensional covariance matrices, Journal of Multivariate Analysis 88 (2) (2004) 365–411. doi:10.1016/S0047-259X(03)00096-4.

[37] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani, L. Wasserman, Distribution-free predictive inference for regression, Journal of the American Statistical Association 113 (523) (2018) 1094–1111. doi:10.1080/01621459.2017.1307116.

[38] F. Zhdanov, Diverse mini-batch active learning, arXiv preprint arXiv:1901.05954 (2019).

[39] O. Sener, S. Savarese, Active learning for convolutional neural networks: A core-set approach, in: International Conference on Learning Representations (ICLR), 2018.

[40] Y. Geifman, R. El-Yaniv, Selective classification for deep neural networks, in: Advances in Neural Information Processing Systems, Vol. 30, 2017.

[41] A. E. Hoerl, R. W. Kennard, Ridge regression: Biased estimation for nonorthogonal problems, Technometrics 12 (1) (1970) 55–67. doi:10.1080/00401706.1970.10488634.

[42] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.

[43] J. H. Friedman, Greedy function approximation: A gradient boosting machine, The Annals of Statistics 29 (5) (2001) 1189–1232. doi:10.1214/aos/1013203451.

[44] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., Scikit-learn: Machine learning in Python, Journal of Machine Learning Research 12 (2011) 2825–2830.

[45] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, New York, 1993.